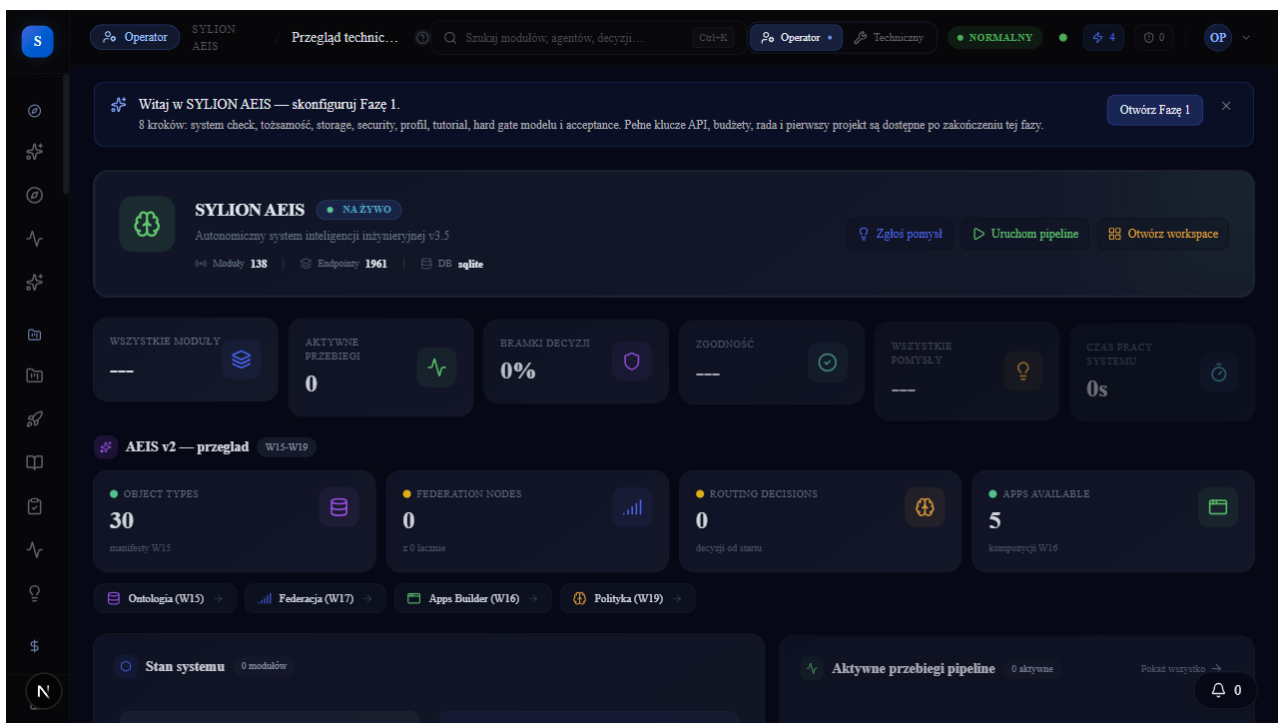


AEIS SYSTEM BOOK 2026

Pełna księga systemu: architektura, moduły, fazy, konfiguracja, funding, testy i screenshoty

Data wygenerowania: 2026-05-17

Zródło: docs/aeis_full_documentation + runtime AEIS 3.5.0



00_overview.png

Zakres obejmuje backend FastAPI, frontend Next.js, menu operatora, lifecycle projektu 16-41, orchestration, memory, skills, funding, mobile, test center i polityke freeze.

Dokument jest snapshotem runtime lokalnego/dev-staging. Elementy produkcyjne, które nadal wymagają hardeningu, są opisane jawnie.

Spis treści księgi

Spis treści księgi	2
00 INDEX	6
AEIS - pełna dokumentacja systemu 2026	6
Spis treści	6
Jak czytać dokumentację	6
Pakiet dokumentacji	6
Najważniejsze fakty runtime	6
Galeria screenshotów	7
Mapa systemu	7
Statusy i legenda	7
Zasada testowania i zamrażania	8
Źródła dowodowe	8
01 START I ARCHITEKTURA	9
Start, runtime i architektura AEIS	9
Spis treści	9
Pierwsze uruchomienie	9
Szybka weryfikacja po starcie	10
Topologia runtime	10
Warstwy W1-W19	10
Source of truth i przepływ danych	11
Mapa zależności	11
Tryby uruchomienia	11
Granice produkcyjne	12
02 MODUŁY API UI	13
Moduły, API i UI AEIS	13
Spis treści	13
Jak liczone moduły	13
Backend packages	13

Najwieksze rodziny API	14
Kluczowe moduly logiczne	14
Menu operatora	15
Tryb techniczny	16
Mapa UI do API	16
Route-only ryzyko	16
03 PROJEKTY FAZY TUTORIAL	18
Projekty, fazy i tutorial operatora	18
Spis tresci	18
Cel lifecycle projektu	18
Tutorial: pierwszy projekt	18
Fazy 16-19 Project Start	19
Fazy 20-25 Council to Ksiega	20
Fazy 26-31 Planning	21
Fazy 32-41 Execution	22
Cztery projekty probne	23
Scenariusze pozytywne i bledne	24
04 KONFIG ORCHESTRACJA TESTY	25
Konfiguracja, orkiestracja i testy AEIS	25
Spis tresci	25
Konfiguracja systemu	25
Model Control i Council	25
Orchestration J1-J9	26
Skills runtime	27
Memory i evidence	28
Test Center i golden tests	29
Zasady testow jak czlowiek	30
Runbook naprawy bledu	31
05 FUNDING MOBILE HELPTIPS	32
Funding, mobile i HelpTipy	32

Spis treści	32
Funding Autopilot	32
Zakładki funding	33
Funding API	33
Bezpieczny external submit	34
Operator mobile	34
HelpTip - mechanizm	35
Katalog HelpTipów kluczowych ekranów	36
Legenda ikon i kontrolek	37
06 ROADMAP PRODUKCYJNA	38
Roadmapa produkcyjna AEIS 2026	38
Spis treści	38
Werdykt startowy	38
Zasady nienegocjowalne	39
Mapa braków produkcyjnych	39
Roadmapa faz R0-R8	40
Backlog wykonawczy P0-P3	45
Testy akceptacyjne dashboardem	46
Zależności architektoniczne	47
Kryterium Production Ready	47
Galeria screenshotów runtime	48
00 overview	48
01 project start	48
02 council to ksiega	48
03 planning	49
04 execution start	49
05 funding	50
06 test center	50
07 human gate	51
08 orchestration	51

09 operator mobile 52

10 workspace 52

11 model council 53

12 skills 53

13 memory 54

00 INDEX

AEIS - pelna dokumentacja systemu 2026

Data: 2026-05-18

Zakres: runtime lokalny AEIS 3.5.0, frontend Next.js, backend FastAPI, dashboard operatorski, projektowy lifecycle, funding, test center, orchestration, memory, skills, Human Gate i warstwa W1-W19.

Status dokumentacji: wersja 1.1, oparta o kod, OpenAPI, runtime health, screenshoty, retest dashboardowy P1-P4 i roadmapa produkcyjna 2026-05-18.

Spis tresci

- 1. Jak czytac dokumentacje
- 2. Pakiet dokumentacji
- 3. Najwazniejsze fakty runtime
- 4. Galeria screenshotow
- 5. Mapa systemu
- 6. Statusy i legenda
- 7. Zasada testowania i zamrazania
- 8. Zrodla dowodowe

Jak czytac dokumentacje

Ta dokumentacja rozdziela trzy rzeczy:

- fakt runtime: cos dziala w uruchomionym backendzie, frontendzie albo API;
- fakt kodowy: modul istnieje i jest podpiety w repo;
- plan docelowy: architektura, ktora wynika z checklisty production-ready, ale nie musi byc jeszcze w pelni zamkniete.

Jezeli jakas funkcja ma route, ale brak pelnego testu akcji, jest opisana jako **ROUTE_ONLY**, **PARTIAL** albo **API_ONLY**. Jezeli przeszla klikany flow dashboardowy, reload proof i drugi pass, jest oznaczona jako **2X_PASS**.

Pakiet dokumentacji

Plik	Co zawiera
01_START_I_ARCHITEKTURA.md	Pierwsze uruchomienie, topologia runtime, warstwy W1-W19, source of truth, mapa zaleznosci.
02_MODULE_API_UI.md	Modul po module: backend packages, API families, frontend routes, menu operatora, statusy powierzchni.
03_PROJEKTY_FAZY_TUTORIAL.md	Tutorial tworzenia projektu, fazy 16-41, cztery projekty probne od latwego do trudnego.
04_KONFIG_ORCHESTRACJA_TESTY.md	Konfiguracja, model council, orchestration J1-J9, skills, memory, test center, freeze policy.
05_FUNDING_MOBILE_HELPTIPS.md	Funding end-to-end, mobile operator, HelpTipy, legenda zakladek i funkcji.
06_ROADMAP_PRODUKCYJNA.md	Roadmapa brakow, napraw, zmian i testow wymaganych do statusu production ready.
screenshots/	Screenshoty z dzialajacego systemu wykonane 2026-05-17.

Najwazniejsze fakty runtime

Element	Wartosc
Backend lokalny	http://127.0.0.1:8010
Frontend lokalny testowy	http://127.0.0.1:3002
Health	status=ok
Version	3.5.0
Runtime modules	138
Runtime endpoints wedlug /health	1961
OpenAPI path templates	1649
OpenAPI methods policzone z paths	1932
DB mode	sqlite
Event mode	sqlite
Frontend route pages	129
Backend route files *_routes.py	127

Galeria screenshotow

Ekran	Route	Screenshot
Przegląd	/overview	overview (screenshots/00_overview.png)
Start projektu	/project-start	project start (screenshots/01_project_start.png)
Deliberacja i Ksiega	/council-to-ksiega	council (screenshots/02_council_to_ksiega.png)
Planowanie	/planning	planning (screenshots/03_planning.png)
Start wykonania	/execution-start	execution (screenshots/04_execution_start.png)
Funding	/funding	funding (screenshots/05_funding.png)
Test Center	/test-center/dashboard	test center (screenshots/06_test_center.png)
Human Gate	/human-gate	human gate (screenshots/07_human_gate.png)
Orchestration	/orchestration	orchestration (screenshots/08_orchestration.png)
Operator mobile queue	/operator-mobile/queue	mobile (screenshots/09_operator_mobile.png)
Workspace	/workspace	workspace (screenshots/10_workspace.png)
Model Council	/model-council	model council (screenshots/11_model_council.png)
Skills	/skills	skills (screenshots/12_skills.png)
Memory	/memory	memory (screenshots/13_memory.png)

Mapa systemu

```
flowchart LR
  Operator["Operator"] --> Frontend["Next.js Operator Console"]
  Frontend --> API["FastAPI AEIS API"]
  API --> Project["Project Lifecycle 16-41"]
  API --> Governance["Governance / Human Gate / Council"]
  API --> Runtime["Workers / Dispatch / Execution"]
  API --> Data["SQLite dev store / Evidence / Audit"]
  API --> Funding["Funding Autopilot"]
  API --> Skills["Skills Registry and Runtime"]
  API --> Memory["Memory and Evidence APIs"]
  API --> Labs["Devices / SDR / Cellular / VPS / Container labs"]
  Project --> Governance
  Project --> Runtime
  Governance --> Data
  Runtime --> Data
  Funding --> Governance
  Skills --> Runtime
  Memory --> Data
```

Statusy i legenda

Status	Znaczenie
LIVE_VERIFIED	Modul ma kod i runtime/API/UI potwierdzają działanie.
2X_PASS	Flow przeszedł dwa razy z dowodem UI/API/reload.
PARTIAL	Funkcja istnieje, ale nie jest domknięta end-to-end.
ROUTE_ONLY	Strona renderuje, ale akcje nie mają pełnego freeze.
API_ONLY	Endpointy działają, ale brak kompletnej powierzchni UI.
UI_ONLY	UI istnieje bez potwierdzonego backendu.
LEGACY	Element historyczny albo pomocniczy, nie główny source of truth.
PLANNED	Plan docelowy bez wystarczającego runtime.
BROKEN	Kod lub UI istnieje, ale test/runtime przeczy oczekiwanemu zachowaniu.

Zasada testowania i zamrażania

Komenda operatorska obowiązująca dla dalszych testów:

Każdy błąd zatrzymuje flow. Najpierw napraw przyczynę, potem wykonaj dwa retesty. Jeżeli oba retesty przejdą, zapisz evidence, oznacz flow jako frozen i dopiero wtedy idź dalej. Toast, sam route albo sam status 200 nie wystarcza. Wymagane są: efekt UI, efekt API, reload proof i brak błędów konsoli.

Ta zasada została zastosowana w retestach:

- P1 Mini CRM Serwisowy - easy;
- P2 Generator Umów i Ofert - medium;
- P3 Funding Assistant Fundacji - hard;
- P4 Platforma Reagowania Kryzysowego - very hard.

Pełny rejestr: ../aeis_manual_tests/12_HUMAN_DASHBOARD_RETEST_2026_05_17.md.

Źródła dowodowe

Źródło	Co potwierdza
/health backendu	Wersja, liczba modułów, liczba endpointów, tryb DB/event.
/openapi.json	Rodziny API, path templates, metody.
src/sylian-pipeline/sylian/api	Routerzy FastAPI i control planes.
src/sylian-frontend/src/app/(app)	Trasy dashboardu operatorskiego i technicznego.
src/sylian-frontend/src/components/layout/AppSidebar.tsx	Menu, sekcje i legenda zakładek.
src/sylian-frontend/src/components/common/HelpTip.tsx	Wspólny mechanizm dymków pomocy.
docs/aeis_manual_tests/12_HUMAN_DASHBOARD_RETEST_2026_05_17.md	Ostatni klikany retest P1-P4.
docs/aeis_repair_v2/*	Historyczne freeze packi, run logi i bug ledgery.

01 START I ARCHITEKTURA

Start, runtime i architektura AEIS

Spis treści

1. Pierwsze uruchomienie
2. Szybka weryfikacja po starcie
3. Topologia runtime
4. Warstwy W1-W19
5. Source of truth i przepływ danych
6. Mapa zależności
7. Tryby uruchomienia
8. Granice produkcyjne

Pierwsze uruchomienie

Minimalny start lokalny:

```
.\scripts\install.ps1
.\scripts\start-server.ps1
.\start_frontend.ps1
```

Co robi instalator:

- wymaga Python **3.11+**;
- tworzy **.venv**, jeżeli brakuje środowiska;
- instaluje zależności z **requirements.txt**, **requirements-lock.txt** albo **requirements-pg.txt**;
- tworzy **.env.generated**, jeżeli plik nie istnieje;
- generuje lokalne sekrety dev: **SYLION_JWT_SECRET**, **SYLION_INTERNAL_API_KEY**, **SYLION_VAULT_SECRET**.

Co robi backend start:

- ustawia **SYLION_DB_PATH=sylian_aeis.db**;
- ustawia **SYLION_ENV=development**;
- ustawia **PYTHONPATH=src\sylian-pipeline**;
- uruchamia **uvicorn sylian.api.app:app --host 127.0.0.1 --port 8010**.

Co robi frontend start:

- wchodzi do **src/sylian-frontend**;
- uruchamia **npm run dev**;
- korzysta z rewrite w **next.config.ts**, jeżeli **NEXT_PUBLIC_API_URL** jest puste;
- w testach dokumentacyjnych działał z jawnie ustawionym **NEXT_PUBLIC_API_URL=http://127.0.0.1:8010**.

Szybka weryfikacja po starcie

```
Invoke-RestMethod http://127.0.0.1:8010/health
Invoke-RestMethod http://127.0.0.1:8010/openapi.json
npm run build
```

Oczekiwany health w tej dokumentacji:

```
{
  "status": "ok",
  "version": "3.5.0",
  "modules": 138,
  "endpoints": 1961,
  "db_mode": "sqlite",
  "event_mode": "sqlite"
}
```

Ostatni build produkcyjny frontendu:

- **npm run build w src/syilion-frontend;**
- wynik: PASS;
- **125/125** stron statycznych wygenerowanych.

Topologia runtime

Warstwa	Technologia	Rola
Operator UI	Next.js 16, React, Tailwind-style classes, lucide icons	Dashboard operatora, dashboard techniczny, projekt lifecycle, funding, test center.
API	FastAPI	Jeden runtime HTTP agregujący moduły AEIS.
Orchestration	Python services + API routes	Routing modeli, council, dispatch, worker registry, repair/test loops.
Storage dev	SQLite	Lokalny stan projektów, funding, audit, events, memory fragments.
Storage prod target	PostgreSQL + backup + vault	Docelowy production-ready kierunek z checklisty.
Evidence	Markdown, JSON, screenshoty, audit ledgers	Zamrożenia testów, bug ledgery, run logi, artefakty projektów.
Workers	Python worker registry/runtime	Rejestracja, heartbeat, dispatch, rebalance, local smoke workers.
Labs	devices, SDR, cellular, VPS, container	Eksperymentalne moduły runtime i infrastruktury.

Warstwy W1-W19

W	Nazwa	Odpowiedzialnosc	Główna powierzchnia
W1	Kanon i polityki	Zasady systemowe, granice, dokumenty kanoniczne.	/policy, /book , docs
W2	Bootstrap	Instalacja, health, start runtime.	/health , scripts
W3	Tozsamosc i role	Operator, role, uprawnienia, profile.	/auth, /roles, /settings/profile
W4	Modele i providerzy	Provider catalog, model registry, budget.	/ai-models, /budget, /secrets
W5	Infrastruktura	Runtime, environments, deploy, VPS, container.	/environments, /runtime, /deploy
W6	Defaulty	Workspace defaults, autonomia, guard defaults.	/workspace-defaults, /autonomy
W7	Governance	Human Gate, D0-D5, rada, gates.	/human-gate, /governance, /gates
W8	Memory	Wiedza, kanon, evidence, retrieval.	/memory, /source-of-truth
W9	Skills	Registry, executor, demand signals.	/skills
W10	Intake	Pomysly, project-start, pipeline.	/workspace, /project-start, /idea-vault
W11	Model Council	Role rady, quorum, głosowanie, rozmowy AI.	/model-council, /orchestration/council-rules
W12	Ksiega i truth	Council to Ksiega, freeze canon/masterplan.	/council-to-ksiega, /evidence-spine

W	Nazwa	Odpowiedzialnosc	Glowna powierzchnia
W13	Advisor i masterplan	Doradca, rekomendacje, planning.	/advisor, /planning
W14	Testy	Test center, golden tests, release gates.	/test-center, /golden-tests
W15	Ontologia	Obiekty domenowe, kontrakty, typy.	/ontology, /contracts, /role-catalog
W16	Execution	Workerzy, build, dispatch, artefakty.	/execution-start, /workers, /apps-builder
W17	Integracje i funding	Funding, devices, federation, labs.	/funding, /devices, /federation
W18	Terminal	Komendy, replay, ownership, rollback.	/terminal, /terminal/replay
W19	Audit i ewolucja	Audit trail, freeze, drift, repair backlog.	/audit, /drift, /rebuild

Source of truth i przepływ danych

AEIS nie ma jeszcze jednego globalnego source-of-truth w stylu produkcyjnego MemoryPlane. W runtime dev prawda jest federacyjna:

- API state w SQLite;
- project lifecycle state;
- audit chain projektu;
- freeze register;
- screenshots;
- JSON/Markdown evidence;
- OpenAPI jako kontrakt runtime;
- dokumenty napraw i retestów.

Docelowa regola production-ready:

Write idzie przez centralny plane domenowy.
Read view moze byc materialized view albo replica.
Kazdy artefakt ma provenance, checksum, retention policy i evidence_id.
Kazda decyzja D3+ ma Human Gate albo Council evidence.

Mapa zaleznosci

```

flowchart TD
    Start["W2 start runtime"] --> Config["W6 konfiguracja"]
    Config --> Intake["W10 intake projektu"]
    Intake --> P16["Fazy 16-19 Project Start"]
    P16 --> P20["Fazy 20-25 Council to Ksiega"]
    P20 --> P26["Fazy 26-31 Planning"]
    P26 --> P32["Fazy 32-41 Execution"]
    P32 --> Audit["W19 audit/evidence/freeze"]

    Governance["W7 Human Gate / Governance"] --> P16
    Governance --> P20
    Governance --> P26
    Governance --> P32
    Model["W4/W11 Model Council"] --> P20
    Model --> P26
    Skills["W9 Skills"] --> P26
    Skills --> P32
    Memory["W8 Memory"] --> P20
    Memory --> P26
    Memory --> Audit
    Funding["W17 Funding"] --> Governance
    Funding --> Intake
    Testy["W14 Test Center"] --> P32
    Testy --> Audit
  
```

Tryby uruchomienia

Tryb	Komenda / plik	Zastosowanie
Root API dev	<code>./scripts/start-server.ps1</code>	Domyslny lokalny backend AEIS.
Frontend dev	<code>./start_frontend.ps1</code>	Next.js operator console.
Standalone legacy dashboard	<code>python dashboard/start.py</code>	Starszy dashboard Python, pomocniczy/legacy.
Docker dashboard stack	<code>src/sylian-pipeline/docker-compose.yml</code>	Dashboard + Redis + Caddy + Prometheus/Grafana.
Full API stack	<code>src/sylian-pipeline/docker-compose.full.yml</code>	PostgreSQL + NATS + API.
PostgreSQL-only stack	<code>src/sylian-pipeline/docker-compose.pg.yml</code>	API z PostgreSQL bez NATS.
Dev overlay	<code>src/sylian-pipeline/docker-compose.dev.yml</code>	Lokalny overlay z MailHog/Adminer/Redis.

Granice produkcyjne

Aktualny runtime dokumentowany tutaj jest lokalny/dev-staging. Najwazniejsze granice przed produkcja:

- PostgreSQL powinien byc wymagany dla staging/production;
- SQLite powinien zostac tylko w development;
- vault powinien zastapic plaintext secrets;
- Human Gate powinien byc jednym globalnym plane;
- Memory powinno miec jeden write plane;
- global deploy pipeline musi miec canary, rollback i DR drill;
- mobile approval wymaga device binding, push i non-repudiation;
- route-only pages musza miec backend action freeze.

02 MODULY API UI

Moduly, API i UI AEIS

Spis tresci

1. Jak liczone moduly
2. Backend packages
3. Najwieksze rodziny API
4. Kluczowe moduly logiczne
5. Menu operatora
6. Tryb techniczny
7. Mapa UI do API
8. Route-only ryzyko

Jak liczone moduly

Snapshot 2026-05-17:

- backend health: **138** modulow runtime;
- OpenAPI: **1649** path templates, **1932** metody policzone po path operations;
- backend route files: **127** plikow ***_routes.py**;
- frontend route pages: **129** plikow **page.tsx**;
- menu glowne: tryb operatora + tryb techniczny + AEIS v2.

Liczba **138** z **/health** jest liczba runtime deklarowana przez aplikacje. Liczba folderow i routerow jest wieksza, bo obejmuje pomocnicze API, legacy, laboratoria, aliasy i integracje.

Backend packages

Package	Rola
aeis, aeis_v2	Rdzen AEIS, test center, terminal, W14-W19, v2 surfaces.
api	FastAPI routers, app bootstrap, middleware, route aggregation.
autonomy	Autonomia, polityki samodzielnosci, konfiguracja decyzji.
cellular, sdr, devices, container, vps	Laboratoria runtime i infrastruktury.
cognitive, providers, grpc, grpc_stubs	Modele, LLM runtime, providerzy, gRPC.
contracts, core, db	Kontrakty, rdzen domenowy, baza danych.
demo	Reference/demo projects.
efficiency, quality, security	Guardy i metryki jakosci, kosztu, security.
execution, pipeline, worker	Wykonanie, pipeline, worker registry, dispatch.
funding_autopilot	Modul grantowy/funding.
governance	Rada, Human Gate, D-level governance.
infra, integration, observability, monitoring	Integracje, runtime truth, logs, metrics, traces.
memory	Kanon, evidence, retrieval, knowledge base.

Package	Rola
operator_mobile	Mobile approval plane.
project_mode	Projekty, lifecycle, W18/project runtime.
rebuild, sim, surface	Rebuild, symulacje, surface modules.
skills	Skills registry, catalog, runtime executor, demand signals.

Najwieksze rodziny API

Top OpenAPI families według liczby metod:

Prefix	Metody	Znaczenie
/api/v1/governance	117	Governance, Human Gate, policy, council-related APIs.
/api/v1/monitoring	90	Monitoring, budgets, metrics, runtime panels.
/api/v1/workspace	73	Workspace AI, sessions, books, project kickoff.
/api/v1/security	62	Security, profiles, audit, scans.
/api/v1/aeis	56	AEIS decomposition/core high-level APIs.
/api/v1/funding	53	Funding Autopilot.
/api/v1/core	51	Core runtime.
/api/v1/reference	51	Reference/demo projects.
/api/v1/cognitive	46	Cognitive/model runtime.
/api/v1/memory	44	Memory, evidence, retrieval.
/api/v1/projects	43	Project registry, lifecycle, details.
/api/v1/skills	37	Skills registry/runtime.
/api/v1/execution	36	Execution APIs.
/api/v1/cellular	34	Cellular lab.
/api/v1/advisor	34	Advisor cards/preferences.
/api/v1/orchestration	32	J-flow orchestration.
/api/v1/execution-start	30	Fazy 32-41.
/api/v1/quality	29	Quality gate APIs.
/api/v1/rebuild	29	Rebuild/self-repair.
/api/v1/workers	29	Worker registry, autoscaler.

Kluczowe moduly logiczne

Modul	UI	API	Status
Bootstrap/Health	/health, /runtime	/health, /api/v1/health, /api/v1/runtime	LIVE_VERIFIED
Workspace AI	/workspace	/api/v1/workspace	PARTIAL_2X_PASS
Pipeline	/pipeline	/api/v1/pipeline	PARTIAL, build fixed
Project Start	/project-start	/api/v1/project-start	2X_PASS P1-P4
Council to Ksiega	/council-to-ksiega	/api/v1/council-to-ksiega	2X_PASS P1-P4
Planning	/planning	/api/v1/planning	2X_PASS P1-P4
Execution Start	/execution-start	/api/v1/execution-start	2X_PASS P1-P4
Human Gate	/human-gate, /operator-mobile/queue	/api/v1/governance, /api/v1/mobile	PARTIAL
Funding	/funding	/api/v1/funding	PARTIAL_2X_PASS, local rehearsal
Skills	/skills	/api/v1/skills	2X_PASS create/execute/signal
Memory	/memory	/api/v1/memory	PARTIAL_ROUTE
Model Council	/model-council, /orchestration/council-rules	/api/v1/council, /api/v1/orchestration	PARTIAL

Modul	UI	API	Status
Workers	<code>/workers</code>	<code>/api/v1/workers</code>	LIVE_VERIFIED
Test Center	<code>/test-center</code> , <code>/test-center/dashboard</code>	<code>/api/v1/test-center</code> , <code>/api/v1/testing</code>	PARTIAL_2X_PASS
Terminal W18	<code>/terminal</code> , <code>/terminal/replay</code>	<code>/api/v1/terminal</code>	project W18 2X_PASS, global partial
Ontology	<code>/ontology</code>	<code>/api/v1/ontology</code>	PARTIAL_ROUTE
Apps Builder	<code>/apps-builder</code>	<code>/api/v1/apps</code>	PARTIAL_ROUTE
Federation	<code>/federation</code>	<code>/api/v1/federation</code>	PARTIAL_ROUTE
Labs	<code>/devices</code> , <code>/sdr</code> , <code>/cellular</code> , <code>/environments/theater</code>	lab prefixes	LAB/PARTIAL

Menu operatora

Sekcje i funkcje z AppSidebar:

Sekcja	Zakladka	Route	Funkcja
Doradca	Centrum dowodzenia	<code>/advisor/cockpit</code>	Widok strategiczny Adwisora.
Doradca	Doradca na zywo	<code>/advisor</code>	Karty doradcze, rekomendacje, decyzje.
Doradca	Monitor projektow	<code>/dashboard/operator-monitor</code>	Monitor aktywnych projektow.
Doradca	Pierwsze uruchomienie	<code>/onboarding</code>	Wizard startowy operatora.
Projekty	Projekty	<code>/projects</code>	Lista i szczegoly projektow.
Projekty	Start projektu	<code>/project-start</code>	Tworzenie projektu, fazy 16-19.
Projekty	Deliberacja i Ksiega	<code>/council-to-ksiega</code>	Fazy 20-25.
Projekty	Planowanie	<code>/planning</code>	Fazy 26-31.
Projekty	Start wykonania	<code>/execution-start</code>	Fazy 32-41.
Projekty	Skarbiec pomyslow	<code>/idea-vault</code>	Pomysly, attach, przejście do projektu.
Finansowanie	Doradca grantow	<code>/funding</code>	Funding Autopilot.
Decyzje	Decyzje	<code>/decisions</code>	Decyzje systemowe i projektowe.
Decyzje	Rada	<code>/governance</code>	Governance/council.
Decyzje	Bramka czlowieka	<code>/human-gate</code>	Queue approve/reject.
Decyzje	Pakiety dowodowe	<code>/evidence</code>	Evidence packs.
Decyzje	Sciezka audytu	<code>/audit</code>	Audit trail.
Testowanie	Centrum testow W14	<code>/test-center</code>	Test center.
Testowanie	Teatr modeli	<code>/test-center/theater</code>	Symulacje i teatr modeli.
Konfiguracja	Ustawienia doradcy	<code>/settings/advisor</code>	Preferencje Adwisora.
Konfiguracja	Modele AI	<code>/ai-models</code>	Providerzy i modele.
Konfiguracja	Domyslny obszar pracy	<code>/workspace-defaults</code>	Defaulty projektow.
Konfiguracja	Guardy	<code>/coherence-guard</code> , <code>/cost-guard</code> , <code>/security-guard</code> , <code>/quality-guard</code> , <code>/provenance-guard</code>	Guard configuration.
Konfiguracja	Szablony	<code>/templates-setup</code>	Templates.
Konfiguracja	Srodowiska	<code>/environments</code>	Environment catalog.
Konfiguracja	Umiejetnosci	<code>/skills</code>	Skills registry/runtime.
Konfiguracja	Budzet modeli	<code>/budget</code>	Model budget.
Konfiguracja	Klucze API	<code>/secrets</code>	Secrets/API keys.
Orkiestracja	Trasy LLM	<code>/orchestration/llm-routing</code>	Routing modeli.
Orkiestracja	Reguly rady	<code>/orchestration/council-rules</code>	Council rules.
Orkiestracja	Audytor	<code>/orchestration/auditor</code>	Auditor agent.
Orkiestracja	Naprawiacze	<code>/orchestration/fixer</code>	Repair agents.
Orkiestracja	Rozdzial pracy	<code>/orchestration/dispatch</code>	Dispatch.

Sekcja	Zakladka	Route	Funkcja
Orkiestracja	Katalog testow	/orchestration/tests	Test catalog.
Orkiestracja	Zespoly	/orchestration/teams	Agent teams.
Orkiestracja	Mapa eventow	/orchestration/event-map	Event map.
Orkiestracja	Rozmowy AI	/orchestration/conversations	AI conversations.
AEIS v2	Warstwy, Ontologia, Terminal, Federation, Policy	/architecture-layers, /ontology, /terminal, /federation, /policy	Zaawansowane planes W7/W15/W17-W19.
Wsparcie	Pomoc i FAQ	/faq	FAQ i runbook.

Tryb techniczny

Tryb techniczny pokazuje dodatkowe powierzchnie dla administratora i developera:

Grupa	Routes
Rdzen	/overview, /pipeline, /workspace, /agents, /modules, /health, /contracts, /performance, /devices, /costs, /sdr, /cellular, /rebuild, /autonomy, /lifecycle, /book
Operacje	/anomalies, /sla, /drift, /risk, /healing, /capacity, /circuits, /golden-tests, /gates, /bundles, /evaluator, /integrations
Bezpieczenstwo	/auth, /roles, /notifications, /connectors, /security-scan

Mapa UI do API

UI	Główny API prefix	Typowe akcje
/project-start	/api/v1/project-start	preview, create, defaults, readiness, edge diagnose.
/council-to-ksiega	/api/v1/council-to-ksiega	convene, verdicts, deliberate, consolidate, generate book, finalize.
/planning	/api/v1/planning	assign models, synthesize skills, generate masterplan/test plan, cost, dry-run.
/execution-start	/api/v1/execution-start	runtime config, initialize, dispatch, phases 34-41, close.
/funding	/api/v1/funding	company profile, calls, ideas, matching, applications, submission, reports.
/skills	/api/v1/skills	register/create, execute, demand signal, catalog, runtime specs.
/memory	/api/v1/memory	kanon, evidence, index, kb, retrieval, self-model.
/operator-mobile	/api/v1/mobile	bind device, queue, decision, approve, reject.
/orchestration/*	/api/v1/orchestration	routing, council, auditor, fixer, dispatch, tests, teams.
/test-center/*	/api/v1/test-center, /api/v1/testing	suites, runs, release gate, simulation, auto-repair.

Route-only ryzyko

AEIS ma duzo stron, ktore renderuja jako dashboard. Produkcyjna dokumentacja traktuje route render jako osobny, slabszy dowod niz action freeze. Dla kazdej strony z akcja mutujaca wymagane jest:

- backend route 2X_PASS;
- frontend action 2X_PASS;
- blad sieci/500/403/timeout opisany i przetestowany;
- **204 No Content** obslugiwane bez falszywego JSON error;
- reload proof;
- brak bladow konsoli.

Priorytet dalszego zamykania: **/advisor**, **/planning**, **/masterplan**, **/source-of-truth**, **/ontology**, **/contracts**, **/templates-setup**, **/environments**.

03 PROJEKTY FAZY TUTORIAL

Projekty, fazy i tutorial operatora

Spis tresci

1. Cel lifecycle projektu
2. Tutorial: pierwszy projekt
3. Fazy 16-19 Project Start
4. Fazy 20-25 Council to Ksiega
5. Fazy 26-31 Planning
6. Fazy 32-41 Execution
7. Cztery projekty probne
8. Scenariusze pozytywne i bledne

Cel lifecycle projektu

Lifecycle projektu AEIS prowadzi operatora od pomyslu do zamknienia projektu. W praktyce najwazniejszy flow to:

Project Start 16-19 -> Council/Ksiega 20-25 -> Planning 26-31 -> Execution 32-41 -> Evidence/freeze

W retestach 2026-05-17 ten flow przeszedl przez cztery projekty probne P1-P4. Kazdy blad po drodze byl naprawiany, testowany dwukrotnie i dopiero potem zamrazany.

Tutorial: pierwszy projekt

1. Uruchom backend i frontend.
2. Otworz **<http://127.0.0.1:3002/project-start>**.
3. Wypelnij nazwe, opis, domeny, budzet, deadline i poziom decyzji.
4. Kliknij preview, sprawdz klasyfikacje i hard blocks.
5. Kliknij create project.
6. Przejdz przez fazy 16-19:
 - default goals;
 - default scope;
 - council defaults;
 - approve readiness.
7. Otworz **[/council-to-ksiega](#)**.
8. Przejdz przez fazy 20-25.
9. Otworz **[/planning](#)**.
10. Przejdz przez fazy 26-31.
11. Otworz **[/execution-start](#)**.

12. Przejdź przez fazy 32-41.

13. Odśwież ekran i sprawdź:

- projekt nadal jest wybrany;
- stan jest **CLOSED**;
- widac **10/10** faz wykonania;
- konsola przeglądarki nie ma błędów;
- API potwierdza acceptance dla faz.

Fazy 16-19 Project Start

API: **/api/v1/project-start**

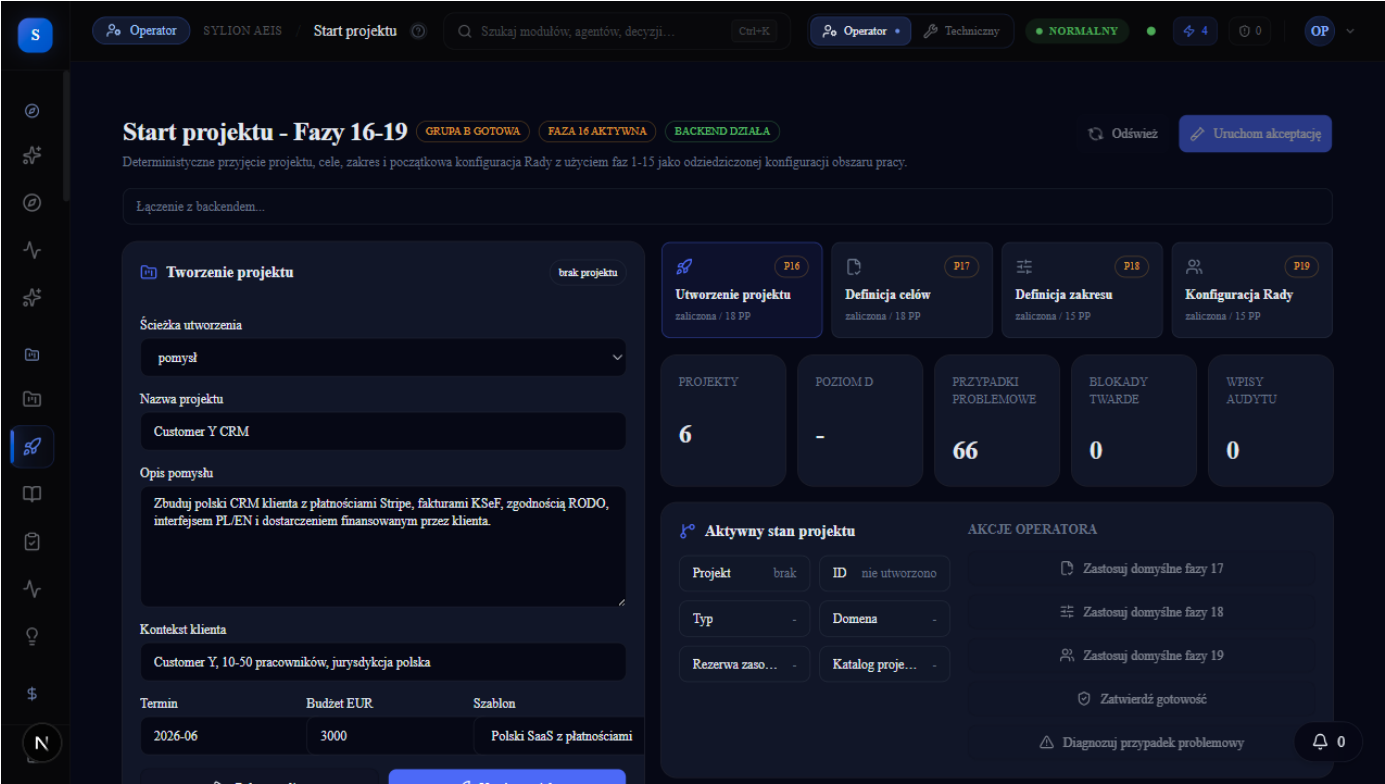
UI: **/project-start**

Faza	Nazwa operatorska	Endpoint / akcja	Co powstaje
16	Intake projektu	POST /projects/preview, POST /projects/create	Projekt, klasyfikacja, domena, D-level, shell lifecycle.
17	Cele i defaulty	POST /projects/{id}/goals/defaults	Cele, KPI, ograniczenia, acceptance goals.
18	Zakres	POST /projects/{id}/scope/defaults	Zakres, runtime constraints, edge cases, risk profile.
19	Gotowosc do rady	POST /projects/{id}/council/defaults, POST /projects/{id}/council/approve-readiness	Council defaults, readiness, przejście do faz 20-25.

Pomocne API:

- **GET /api/v1/project-start;**
- **GET /api/v1/project-start/templates;**
- **GET /api/v1/project-start/projects;**
- **GET /api/v1/project-start/active;**
- **GET /api/v1/project-start/projects/{id};**
- **GET /api/v1/project-start/projects/{id}/phases/{phase}/acceptance;**
- **GET /api/v1/project-start/projects/{id}/edge-cases;**
- **POST /api/v1/project-start/projects/{id}/edge-cases/diagnose.**

Screenshot:



01_project_start.png

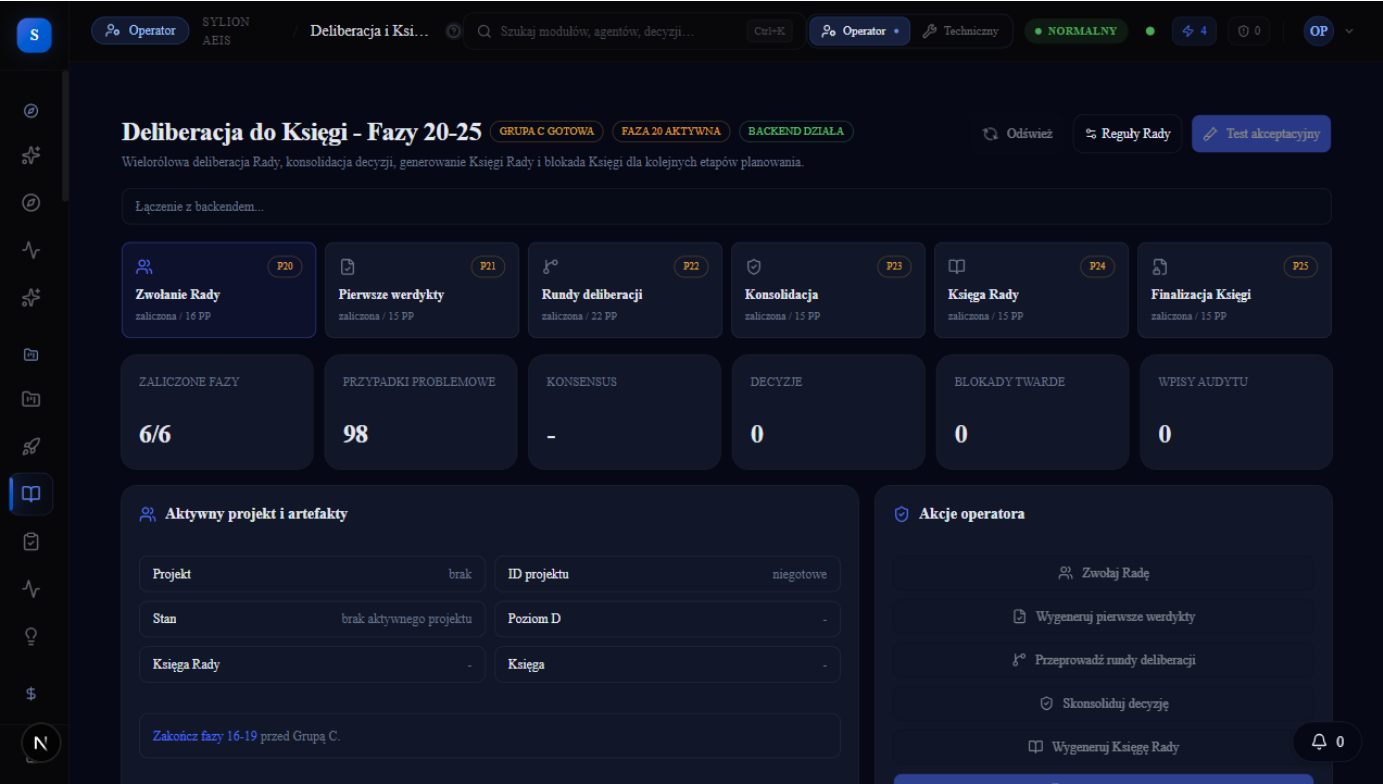
Fazy 20-25 Council to Ksiega

API: /api/v1/council-to-ksiega

UI: /council-to-ksiega

Faza	Nazwa	Endpoint	Wynik
20	Zwołanie Rady	POST /projects/{id}/phase20/convene	Role rady, pytania, kontekst.
21	Pierwsze werdykty	POST /projects/{id}/phase21/initial-verdicts	Werdykty modeli, poziomy konsensusu.
22	Rundy deliberacji	POST /projects/{id}/phase22/deliberate	Rundy dyskusji, sporne pytania, poprawki.
23	Konsolidacja	POST /projects/{id}/phase23/consolidate	Final decisions, risk register, scope guard.
24	Generowanie Ksiega Rady	POST /projects/{id}/phase24/generate-book	Markdown/PDF book, checksum.
25	Finalizacja Ksiegi	POST /projects/{id}/phase25/finalize-ksiega	Final source book i przejście do planning.

Screenshot:



02_council_to_ksiega.png

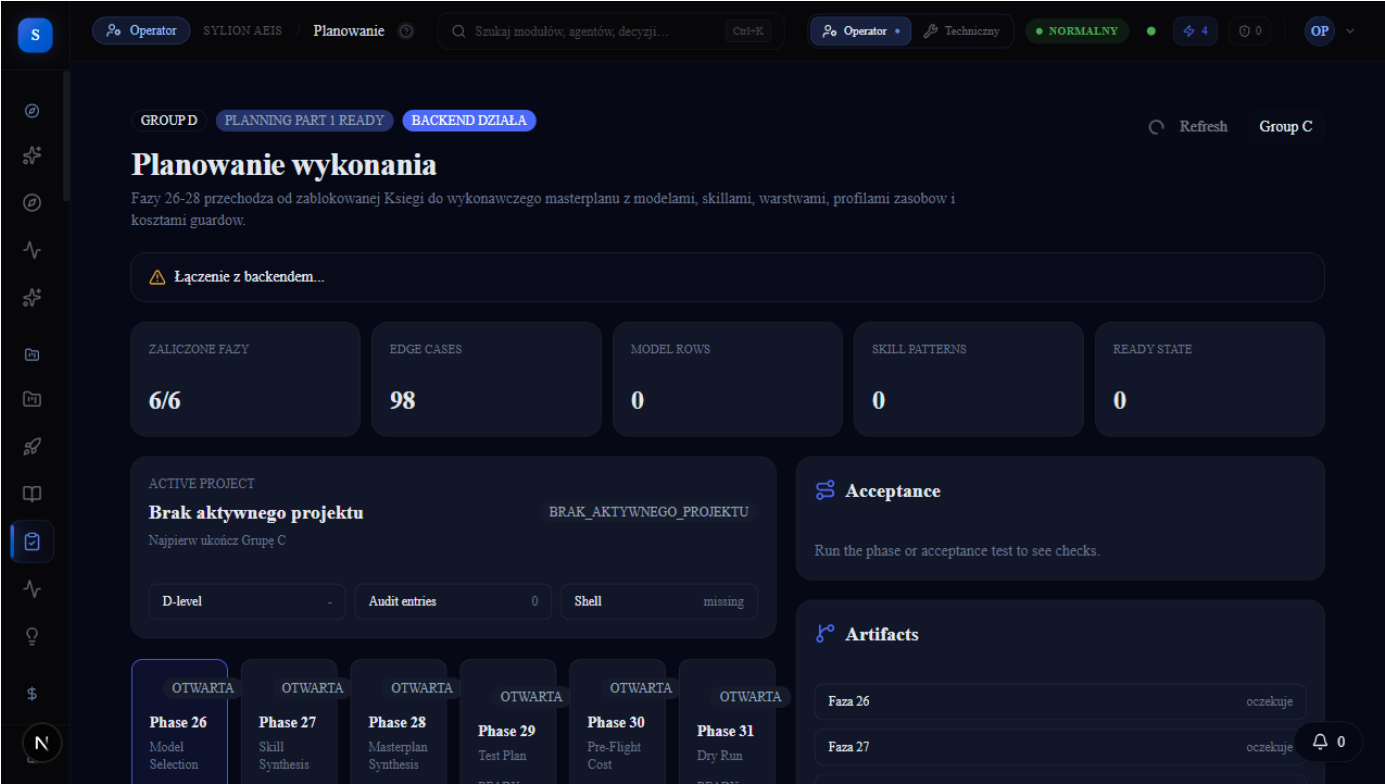
Fazy 26-31 Planning

API: /api/v1/planning

UI: /planning

Faza	Nazwa	Endpoint	Wynik
26	Model Selection	POST /projects/{id}/phase26/assign-models	Model assignment, role, fallback chains, koszt.
27	Skill Synthesis	POST /projects/{id}/phase27/synthesize-skills	Skills projektowe i systemowe.
28	Masterplan Synthesis	POST /projects/{id}/phase28/generate-masterplan	Warstwy wykonania, harmonogram, zaleznosci.
29	Test Plan Synthesis	POST /projects/{id}/phase29/generate-test-plan	L1-L5, human-like tests, acceptance criteria.
30	Pre-Flight Cost Preview	POST /projects/{id}/phase30/preflight-cost	Koszt, limity, guards.
31	Pre-Flight Dry Run	POST /projects/{id}/phase31/dry-run	Gotowosc do build, local-first constraints.

Screenshot:



03_planning.png

Fazy 32-41 Execution

API: /api/v1/execution-start

UI: /execution-start

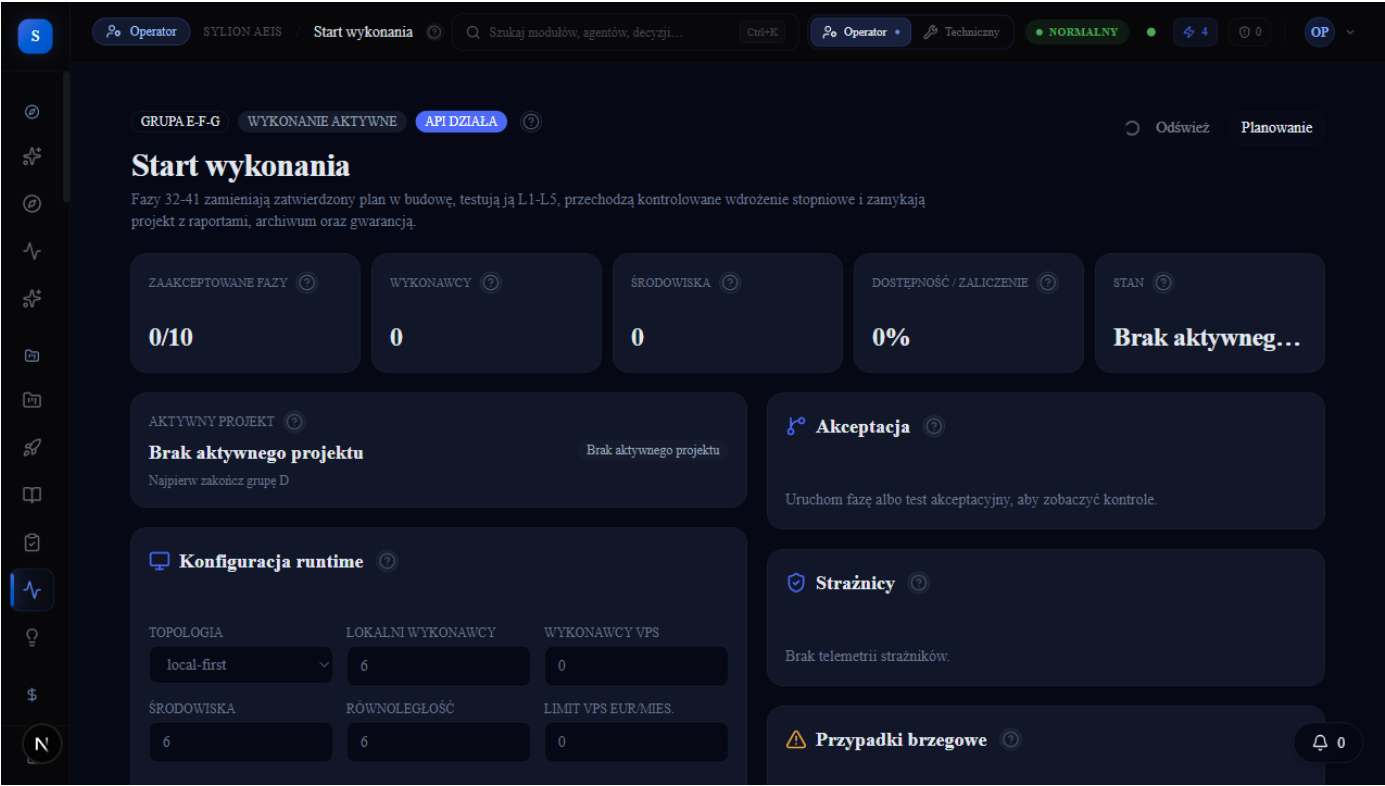
Faza	Nazwa	Endpoint	Wynik
32	Inicjalizacja budowy	POST /projects/{id}/phase32/initialize-build	Workspace, branches, workers, environments, monitoring.
33	Sekwencyjne wykonanie	POST /projects/{id}/phase33/start-execution	Petla budowy, dispatch control, progress.
34	Rada w trakcie budowy	POST /projects/{id}/phase34/reconvene-council	Mid-build council i decision evidence.
35	Orkiestracja budowy	POST /projects/{id}/phase35/activate-orchestration	Koordinacja workerow, recovery, coherence.
36	Zamkniecie budowy	POST /projects/{id}/phase36/complete-build	Artefakty, koszt, worker decommission.
37	Bramki jakosci	POST /projects/{id}/phase37/run-quality-gates	L1-L5, coverage, critical findings, PASS verdict.
38	Akceptacja klienta	POST /projects/{id}/phase38/complete-acceptance	Staging, feedback, signoff.
39	Finalna kontrola	POST /projects/{id}/phase39/authorize-production-deploy	Rollback, monitoring, hard gate.
40	Deploy/proba lokalna	POST /projects/{id}/phase40/execute-production-deploy	Canary/local rehearsal, external calls blocked.
41	Zamkniecie projektu	POST /projects/{id}/phase41/close-project	Raporty, archiwum, warranty, final state CLOSED.

Kontrole dodatkowe:

- runtime configuration: **GET/POST /projects/{id}/runtime-configuration;**
- live workers: **GET/POST /phase32/live-spawn-workers, POST /phase32/stop-live-workers;**

- dispatch: **GET /phase33/dispatch-control**, **POST pause/resume/cancel**;
- W18 commands: **GET /projects/{id}/w18-commands**;
- audit truth map: **GET/POST /audit-truth-map**.

Screenshot:



04_execution_start.png

Cztery projekty probne

Projekt	Trudnosc	D-level	Domena	Cel testowy	Wynik
P1 Mini CRM Serwisowy	Easy	D3	crm	Najprostszy lokalny system CRM.	CLOSED, 10/10 , local rehearsal.
P2 Generator Umow i Ofert	Medium	D3	crm	Dokumenty, oferty, workflow biznesowy.	CLOSED, 10/10 .
P3 Funding Assistant Fundacji	Hard	D4	funding	Funding workflow bez realnego external submit.	CLOSED, 10/10 , local rehearsal.
P4 Platforma Reagowania Kryzysowego	Very hard	D5	aeis_multi_domain	Multi-domain, governance, local-first, no external calls.	CLOSED, 10/10 , reload-proof.

P4 reload-proof:

- route: **/execution-start**;
- UI: **PROJEKT ZAMKNIETY**;
- accepted phases: **10/10**;
- console errors: **0**;
- API state: **CLOSED**;
- deploy mode: **local_release_rehearsal_no_external_calls**.

Scenariusze pozytywne i bledne

Scenariusz pozytywny

1. Projekt tworzony jest przez UI.
2. Kazda grupa faz ma widoczny stan i acceptance.
3. Kazda akcja mutujaca wraca z backendu.
4. Po reloadzie stan pozostaje.
5. API potwierdza stan projektu.
6. Konsola przegladarki nie ma bledow.
7. Flow zostaje zapisany w evidence.

Scenariusze bledne

Blad	Objaw	Reakcja operatora
Failed to fetch po preflight	UI kasuje projekt albo pokazuje falszywy offline.	Zatrzymac flow, sprawdzic OPTIONS/GET, naprawic retry/critical load, 2x retest.
Toast bez zapisu	UI pokazuje sukces, reload cofa stan.	Nie zamrazac, sprawdzic API i persisted state.
Brak Human Gate	D4/D5 akcja przechodzi bez decyzji.	Blokuje freeze governance.
Brak provenance	Memory/funding/test result nie ma zrodla.	Blokuje freeze evidence.
Deploy external bez zgody	Phase 40 wykonuje external call.	Krytyczny blocker, wymagany Human Gate D5.
Konsola ma blad JS	UI moze dzialac pozornie, ale runtime jest niespojny.	Naprawic i wykonac dwa reload-proofy.

04 KONFIG ORCHESTRACJA TESTY

Konfiguracja, orkiestracja i testy AEIS

Spis tresci

- 1. Konfiguracja systemu
- 2. Model Control i Council
- 3. Orchestration J1-J9
- 4. Skills runtime
- 5. Memory i evidence
- 6. Test Center i golden tests
- 7. Zasady testow jak czlowiek
- 8. Runbook naprawy bledu

Konfiguracja systemu

Najwazniejsze powierzchnie konfiguracji:

Powierzchnia	Route	Co konfiguruje
Workspace defaults	/workspace-defaults	Defaulty projektu, autonomia, budżet, testy, acceptance.
AI Models	/ai-models	Providerzy, modele, klucze, testy providerow.
Budget	/budget	Limity kosztow modeli i runtime.
Secrets	/secrets	Klucze API i sekrety. Docelowo vault, nie plaintext.
Environments	/environments	Srodowiska lokalne, staging, theater, runtime constraints.
Templates	/templates-setup	Szablony projektow, dokumentow, pipeline.
Guards	/coherence-guard, /cost-guard, /security-guard, /quality-guard, /provenance-guard	Guard policies i thresholds.
Autonomy	/autonomy	Poziomy autonomii i blokady decyzji.
Policy	/policy	Globalne policy W19.

W praktyce operator powinien zaczac od:

- 1. **/onboarding** - pierwszy wizar;
- 2. **/settings/advisor** - preferencje doradcy;
- 3. **/workspace-defaults** - defaulty projektow;
- 4. **/ai-models** i **/budget** - modele i koszt;
- 5. **/secrets** - klucze;
- 6. **/guards** albo konkretne guard pages;
- 7. **/project-start** - pierwszy projekt.

Model Control i Council

AEIS ma kilka powierzchni modelowych:

- **/ai-models** - konfiguracja providerow i modeli;
- **/model-council** - widok rady modeli;
- **/orchestration/llm-routing** - routing modeli do zadan;
- **/orchestration/council-rules** - reguly rady;
- **/orchestration/conversations** - rozmowy i glosy modeli;
- **/budget** - koszt i limity.

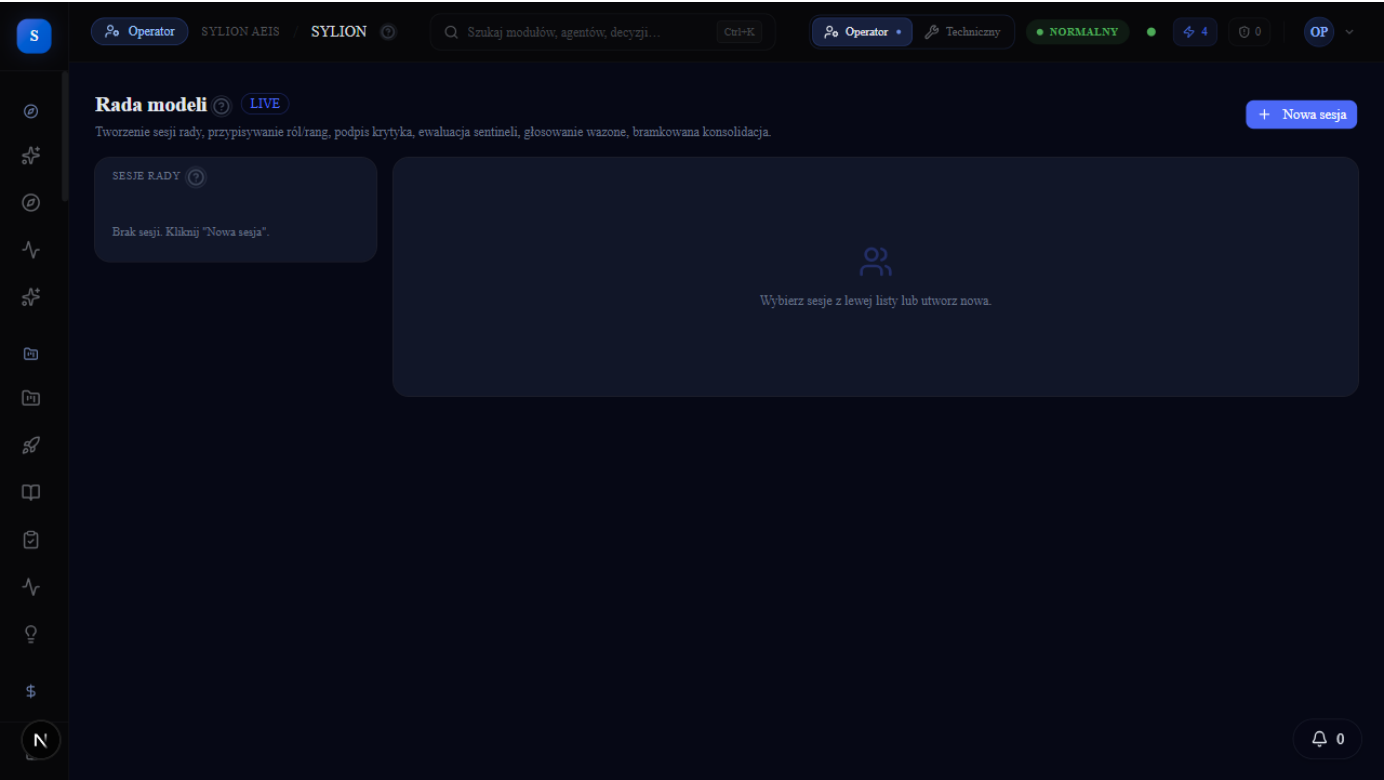
Docelowa regola:

Kazdy model uzyty w systemie musi istniec w ModelRegistry.
Council settings musza wskazywac ModelRegistry.model_id.
Routing musi przejsc przez BudgetEnforcer.
Key rotation musi propagowac sie do wszystkich planes.

Stan runtime:

- routing i council dzialaja w wybranych flow;
- planning phase 26 generuje model assignment;
- P4 wygenerowal **22** model rows;
- globalny jeden **ModelControlPlane** nadal jest celem architektonicznym, nie w pelni zamknietym faktem.

Screenshot:



11_model_council.png

Orchestration J1-J9

Orchestration dzieli prace modeli i agentow na konkretne planes:

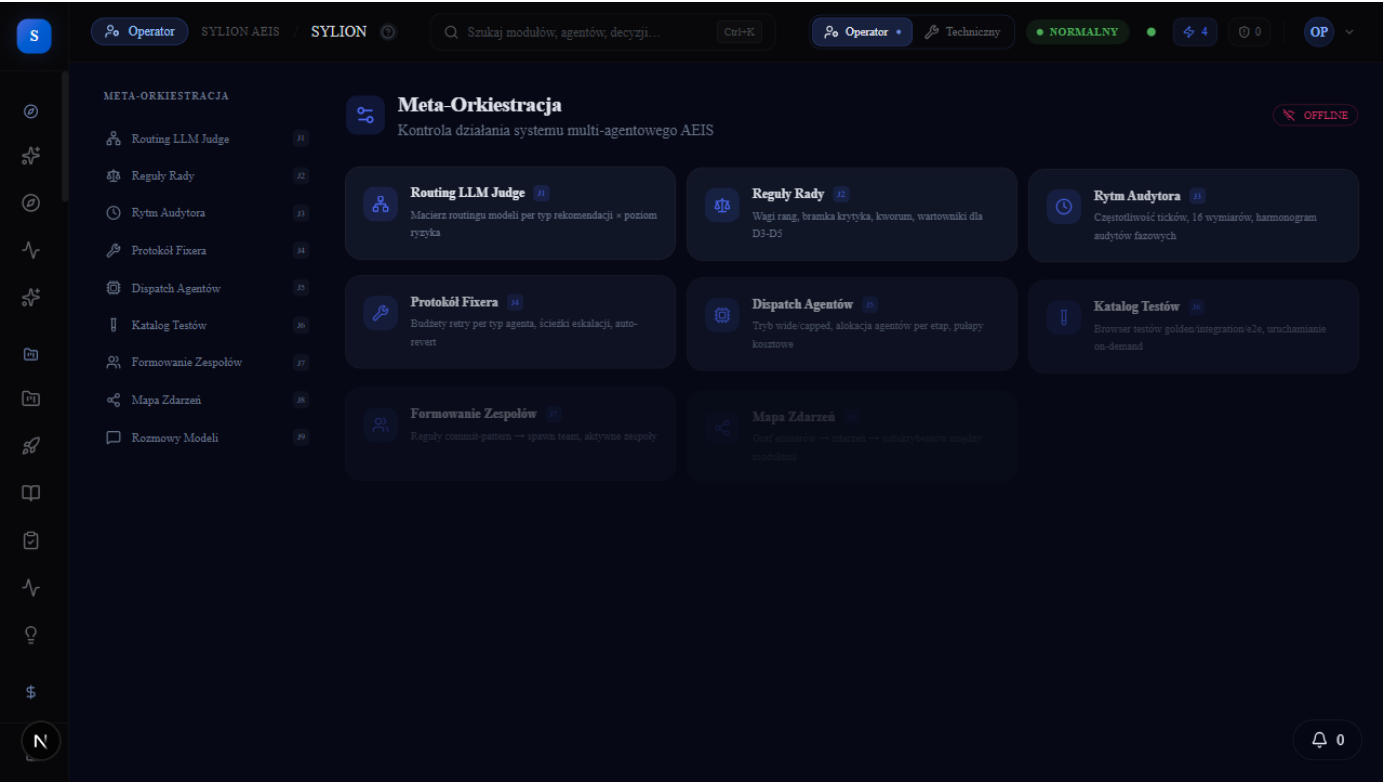
J	Powierzchnia	Funkcja
J1	/orchestration/llm-routing	Wybor modeli, fallback, routing.

J	Powierzchnia	Funkcja
J2	/orchestration/council-rules	Quorum, role, reguly rady.
J3	/orchestration/auditor	Audytora i gate checks.
J4	/orchestration/fixer	Naprawiacze i auto-repair.
J5	/orchestration/dispatch	Rozdzial pracy, target worker/model.
J6	/orchestration/tests	Katalog testow i golden tests.
J7	/orchestration/teams	Zespoly agentow.
J8	/orchestration/event-map	Event map i telemetry.
J9	/orchestration/conversations	Rozmowy AI, decyzje, trace.

W execution phase 33 dispatch control ma osobne operacje:

- GET /api/v1/execution-start/projects/{id}/phase33/dispatch-control;
- POST /pause-dispatch;
- POST /resume-dispatch;
- POST /cancel-dispatch.

Screenshot:



08_orchestration.png

Skills runtime

API skills ma kilka warstw:

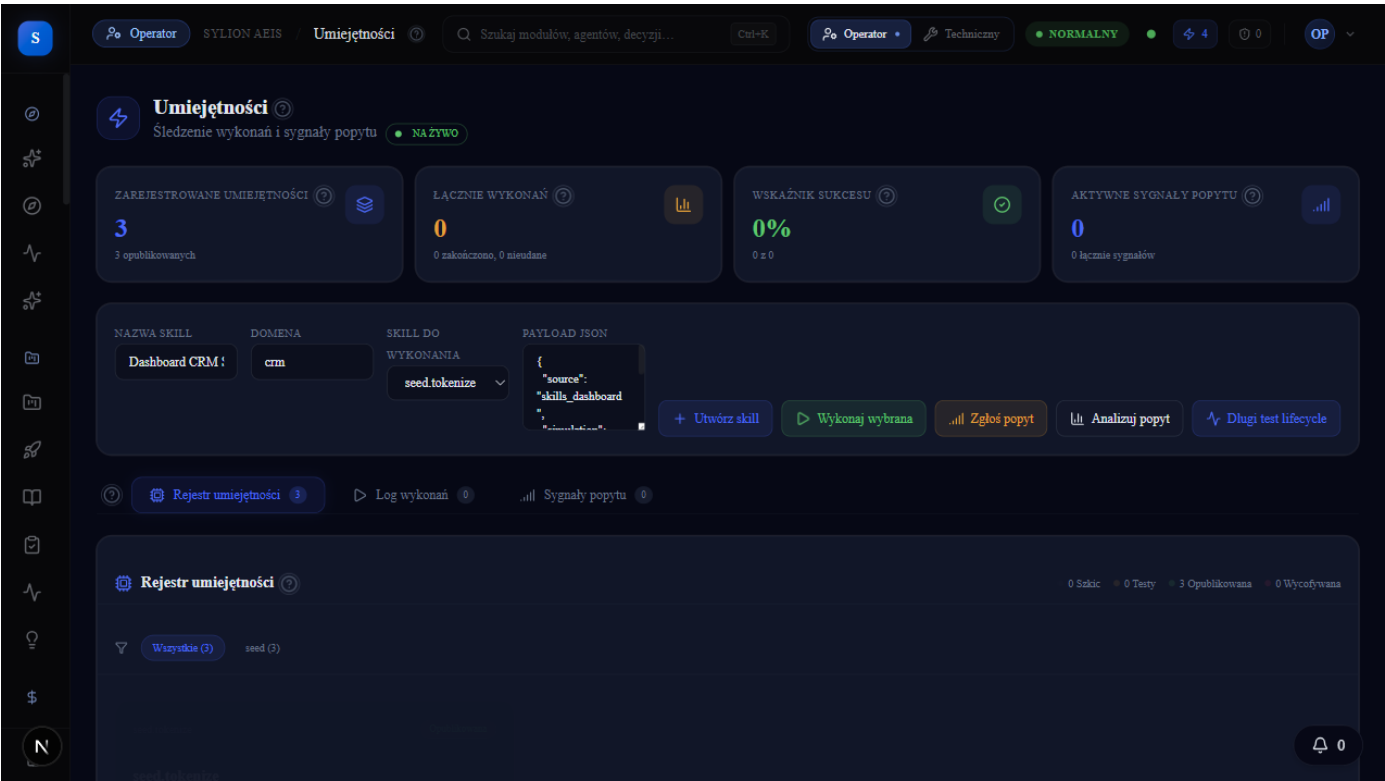
Warstwa	Endpointy	Znaczenie
Health/list	GET /api/v1/skills/health, GET /api/v1/skills	Stan i lista.
Legacy register	POST /api/v1/skills/register	Rejestracja zgodna z poprzednim API.
Demand	POST /demand/signals, GET /demand/signals, POST /demand/analize	Sygaly zapotrzebowania.

Warstwa	Endpointy	Znaczenie
Executions	POST /executions, GET /executions	Uruchomienia skills.
Registry	POST /skills, GET /skills, POST /skills/{id}/publish/deprecate/retire	Lifecycle skill.
Catalog	POST /catalog, GET /catalog, GET /catalog/recommend	Katalog i rekomendacje.
Runtime	GET /runtime/specs, POST /runtime/execute, GET /runtime/stats	Runtime executor.

Stan:

- create/execute/demand signal były testowane jako **2X_PASS**;
- integracja z pipeline W10 i execution W16 jest nadal czesciowo rozdzielona;
- docelowo **SkillIntegrationLayer** powinien łączyc pipeline step, dispatch i demand signals.

Screenshot:



12_skills.png

Memory i evidence

API memory obejmuje:

Obszar	Endpointy	Funkcja
Obsidian connector	/obsidian/connector, /obsidian/sync, /obsidian/status, /obsidian/graph	Sync notatek i grafu.
Kanon	/kanon/load, /kanon/sections, /kanon/search, /kanon/full-text	Kanoniczna dokumentacja i wyszukiwanie.
Compact	/compact, /compact/record, /compact/fidelity, /compact/stats	Kompakcja i fidelity.
Evidence	/evidence, /evidence/stats, /evidence-store, /evidence/{id}	Dowody i store.
Index	/index/sections, /index/search, /index/stats	Indeks i search.

Obszar	Endpointy	Funkcja
KB	<code>/kb/sources</code> , <code>/kb/query</code> , <code>/kb/stats</code>	Knowledge base sources.
Retrieval	<code>/retrieval</code> , <code>/retrieval/context</code>	Retrieval context.
Self-model	<code>/self-model/*</code>	Model wiedzy o systemie.
Stats/recent	<code>/stats</code> , <code>/recent</code>	Podsumowanie memory.

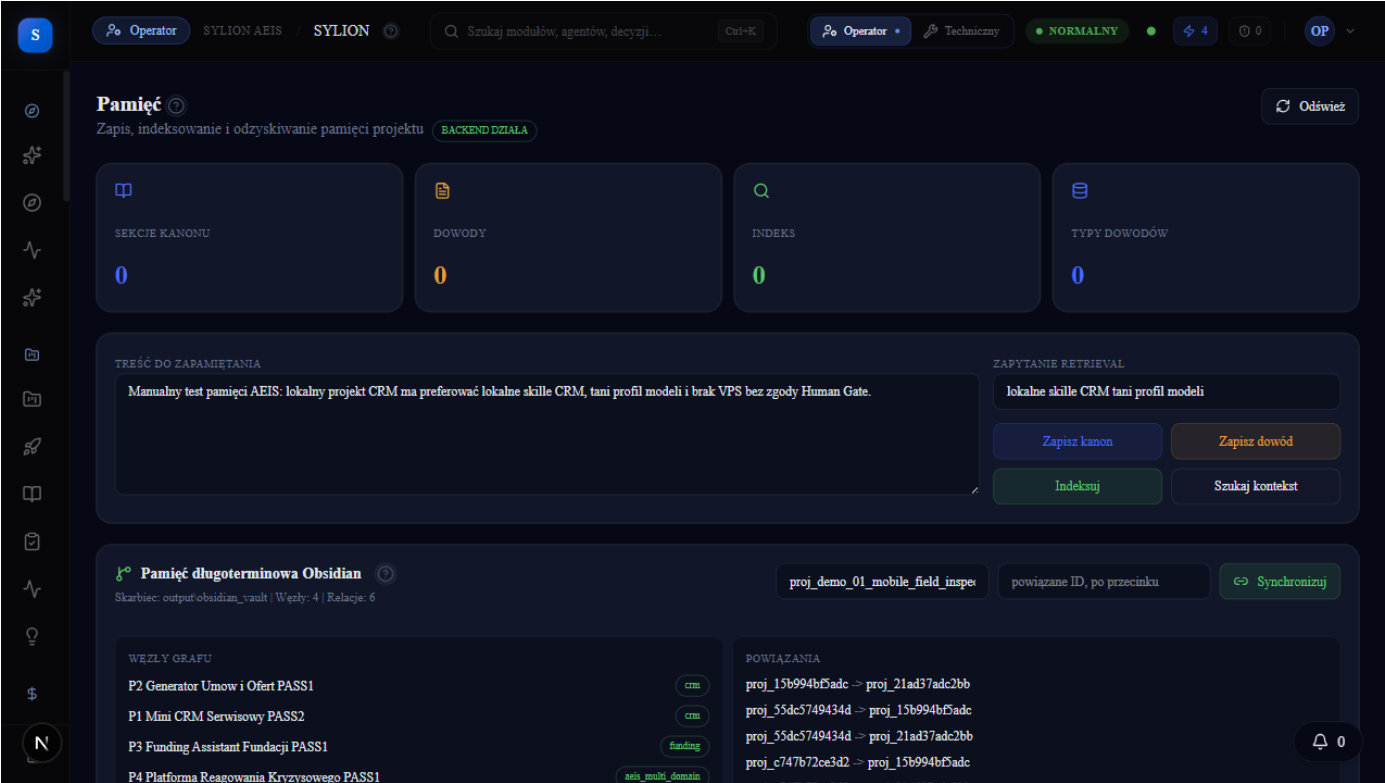
Regola docelowa:

Kazdy wynik memory search ma `project_id`, `provenance`, `evidence_id` i `permission context`.
Per-project DB moze byc read view, ale nie primary write source.

Stan:

- UI `/memory` renderuje i ma screenshot;
- API memory jest szerokie;
- globalny jeden write plane pozostaje luka architektoniczna.

Screenshot:



13_memory.png

Test Center i golden tests

Główna powierzchnia:

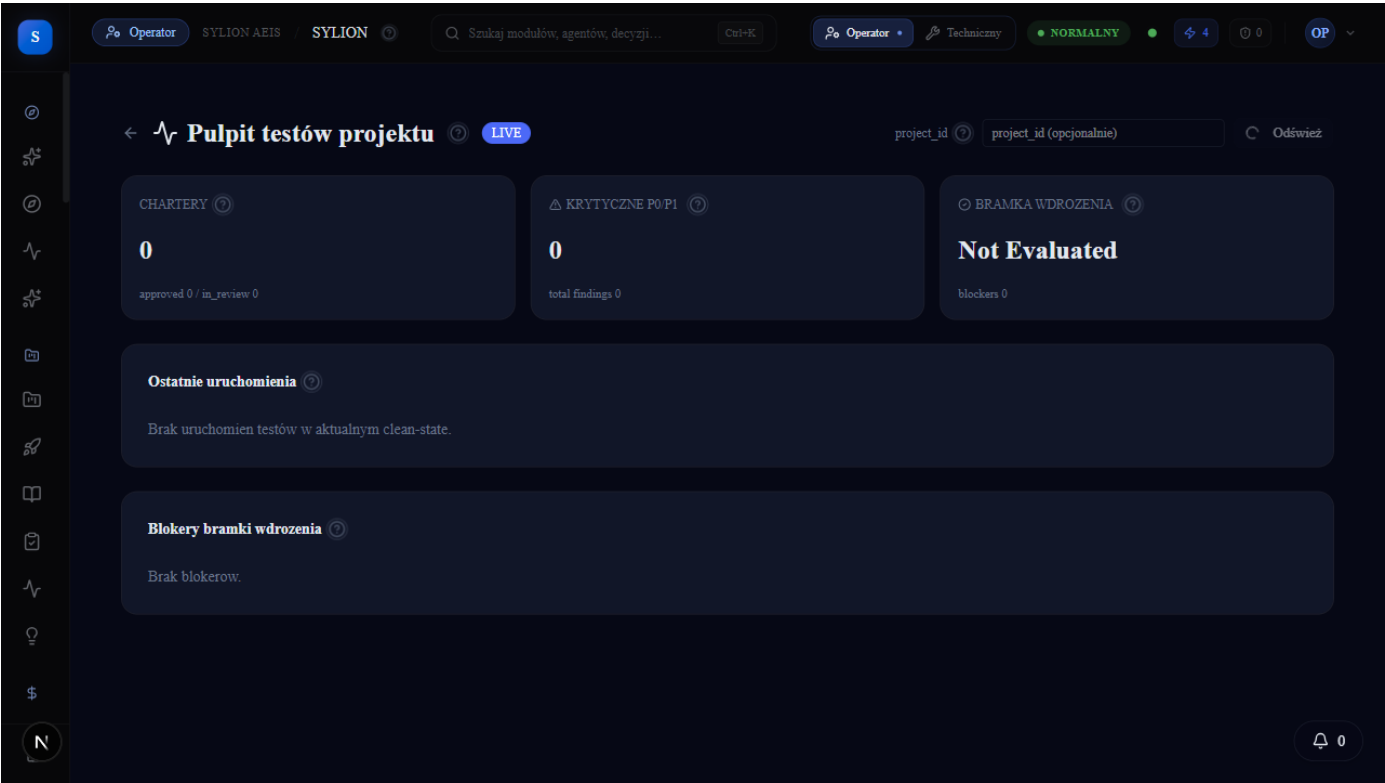
- `/test-center`;
- `/test-center/dashboard`;
- `/test-center/catalog`;
- `/test-center/human-lab`;
- `/test-center/no-mock-scan`;

- /test-center/release-gate;
- /test-center/simulation;
- /test-center/theater;
- /test-center/truth-alignment;
- /golden-tests.

Testy w AEIS dziela sie na:

Poziom	Co obejmuje
L1	Unit tests i izolowane funkcje.
L2	Integracje API/service.
L3	E2E dashboard/API.
L4	Performance/load.
L5	Human-like UI scenarios, klikane przez operatora lub automaty browser.

Screenshot:



06_test_center.png

Zasady testow jak czlowiek

Kazdy test dashboardu powinien udowodnic:

- co operator widzi;
- co operator klika;
- jakie dane wpisuje;
- jaka odpowiedz backendu powstaje;
- czy stan trwa po reloadzie;
- czy po drugim przebiegu wynik jest taki sam;

- czy konsola przeglądarki jest bez błędów.

Minimalny protokół:

1. Otwórz ekran.
2. Sprawdź health/API status.
3. Wpisz dane jak człowiek.
4. Kliknij jedną akcję.
5. Sprawdź UI po akcji.
6. Sprawdź API lub stan projektu.
7. Odśwież ekran.
8. Powtórz pass.
9. Zapisz screenshot, project_id, endpointy i wynik.

Runbook naprawy błędu

```
ERROR -> STOP FLOW
-> opisz błąd
-> znajdź przyczynę w UI/API/logach
-> napraw minimalnie
-> uruchom lint/build/test adekwatny do zmiany
-> wykonaj PASS 1
-> wykonaj PASS 2
-> zapisz evidence
-> oznacz frozen
-> przejdź dalej
```

Nie wolno:

- zamrażać po samym toaście;
- zamrażać po samym **200**;
- automatycznie retry'ować mutujących **POST** bez idempotency key;
- wykonywać realnego external submit/deploy bez Human Gate;
- mieszać danych projektów w memory/search;
- ignorować błędów konsoli.

05 FUNDING MOBILE HELPTIPS

Funding, mobile i HelpTipy

Spis tresci

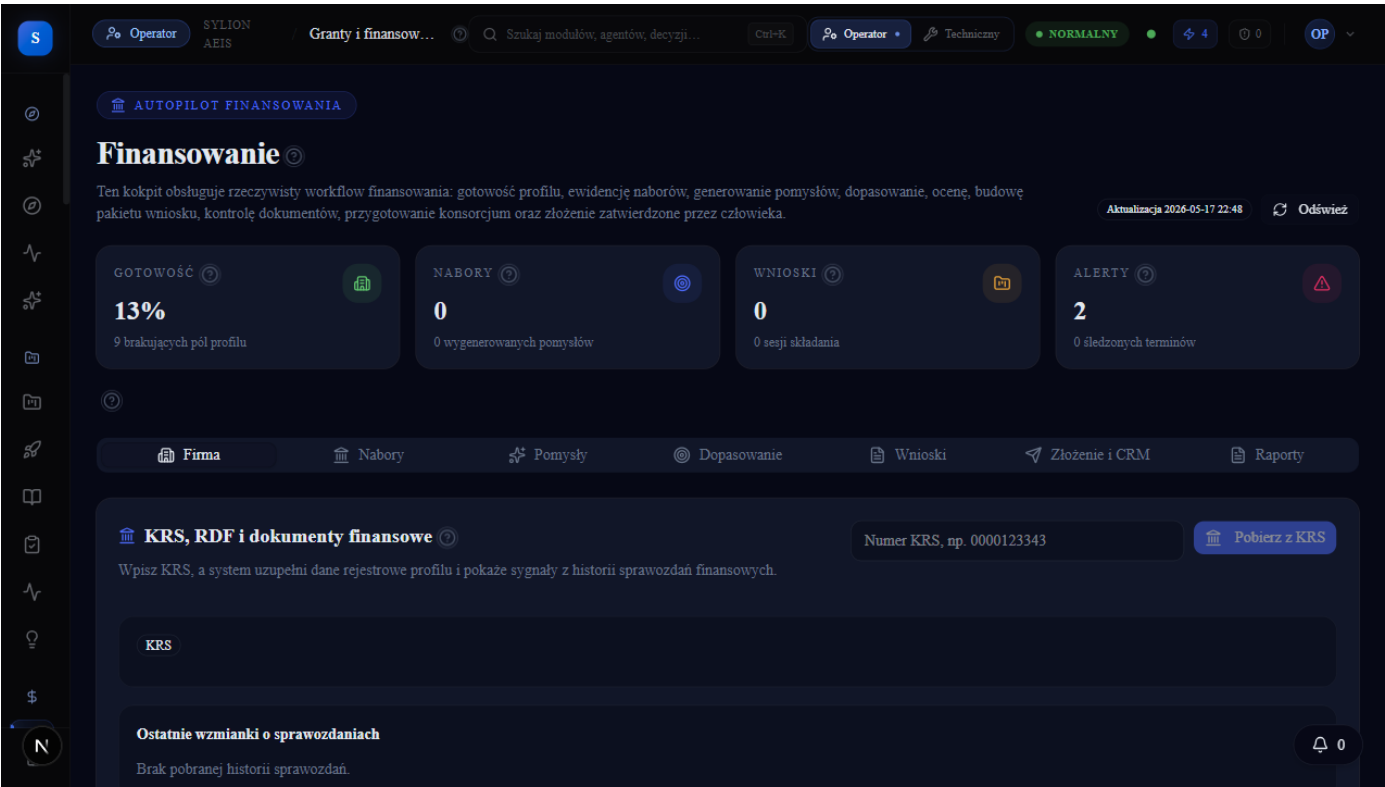
1. Funding Autopilot
2. Zakladki funding
3. Funding API
4. Bezpieczny external submit
5. Operator mobile
6. HelpTip - mechanizm
7. Katalog HelpTipow kluczowych ekranow
8. Legenda ikon i kontrolek

Funding Autopilot

Funding jest realnym pionem domenowym, a nie tylko promptem. Ma:

- backend **syllion/funding_autopilot/routes.py**;
- service/store/schema/permissions;
- UI **/funding**;
- profile firmy;
- zrodla i nabory;
- pomysly projektow;
- matching i scoring;
- aplikacje grantowe;
- submission session;
- CRM tracking;
- raporty i alerty.

Screenshot:



05_funding.png

Zakładki funding

Zakładka	Cel	Typowe akcje
Firma	Profil firmy, dokumenty, KRS, readiness.	Zapis profilu, registry sync, dokumenty, state aid.
Nabory	Programy i call'e.	Lista sources, create programme, create call, scan/search.
Pomysly	Idee projektow grantowych.	Generate ideas, convert idea to project.
Dopasowanie	Matching/scoring/consortium.	Run matching, eligibility, scoring, partner search, outreach.
Wnioski	Aplikacje.	Create application, review, export, document checklist.
Submission	Bramka zlozenia.	Prepare, fill mapping, save draft, request approval, submit receipt.
Raporty	Executive reporting.	Deadlines, alerts, executive report, exports.

Funding API

Najwazniejsze endpointy:

Obszar	Endpointy
Health	GET /api/v1/funding/health
Company	GET/PUT /company-profile, /company-profile/readiness, /documents, /state-aid, /registry-sync
Sources/programmes	GET /sources, GET/POST /programmes, GET/POST /calls, POST /calls/search, GET /calls/scored
Ideas	GET /ideas, POST /ideas/generate, POST /ideas/{id}/convert-to-project
Projects/matching	GET/POST /projects, POST /matching/run, GET /matching/results/{project_id}
Eligibility/scoring	POST /eligibility/check, POST /scoring/run, GET /scoring/{project_id}
Consortium	POST /consortium/analyze, /partners/search, /partners/shortlist, /outreach/generate
Application	POST /application/create, GET /application/{id}, /documents, /review, /export
Submission	POST /submission/prepare, /fill, /save-draft, /request-approval, /submit, GET /submission/receipt, /sessions, /approvals
CRM/reporting	GET /crm/applications, /deadlines, /alerts, /reports/executive

Obszar	Endpointy
Scan	POST /scan/trigger, GET /scan/status/{job_id}

Bezpieczny external submit

W testach P3 funding pozostal local rehearsal. Regola:

External submit = D4.
 Realny submit wymaga Human Gate.
 Preview musi pokazac dokladny payload/PDF.
 Submit zapisuje payload_hash, operator_id, timestamp i receipt.
 Po realnym submit nie ma rollbacku, jest tylko korekta/wycofanie zgodnie z portalem zewnetrznym.

Wymagane pre-submit checks:

- deadline OK;
- source OK;
- legal OK;
- budget OK;
- documents OK;
- review readiness OK;
- operator approval present;
- portal/reference captured;
- CRM entry created.

Operator mobile

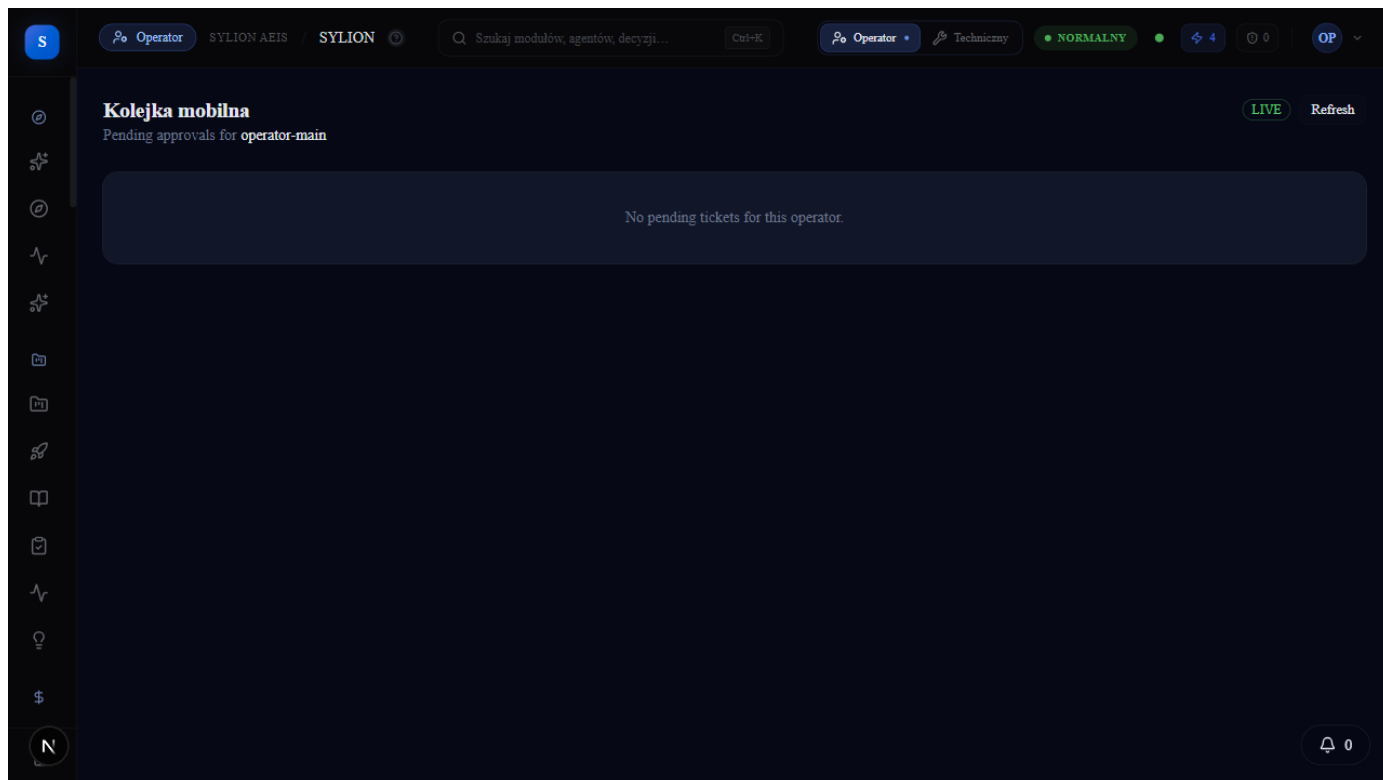
Powierzchnie:

- **/operator-mobile;**
- **/operator-mobile/queue;**
- **/operator-mobile/queue/{ticketId};**
- **/operator-mobile/devices.**

API:

- **POST /api/v1/mobile/devices/bind;**
- **GET /api/v1/mobile/devices;**
- **DELETE /api/v1/mobile/devices/{device_id};**
- **GET /api/v1/mobile/queue;**
- **GET /api/v1/mobile/queue/{ticket_id};**
- **POST /api/v1/mobile/queue/{ticket_id}/decision;**
- **POST /api/v1/mobile/queue/{ticket_id}/approve;**
- **POST /api/v1/mobile/queue/{ticket_id}/reject.**

Screenshot:



09_operator_mobile.png

Status:

- queue view istnieje;
- approve/reject API istnieje;
- docelowo wymagane sa device binding, push, biometric/PIN, non-repudiation i offline queue.

HelpTip - mechanizm

Komponent: **src/sylation-frontend/src/components/common/HelpTip.tsx**.

Zachowanie:

- renderuje okragly znak pomocy;
- otwiera tooltip po hover/focus/click;
- ma **aria-label**;
- przelicza pozycje przy scroll/resize;
- uzywa portalu do **document.body**;
- maksymalna szerokosc tooltipa to **min(420px, viewport - 32px)**;
- automatycznie zmienia strone, gdy brakuje miejsca;
- zamyka tooltip po outside pointer albo **Escape**.

Regola pisania HelpTipow:

- tekst prosty, po polsku;
- 1-3 zdania;
- opisuje ryzyko albo sens funkcji, nie marketing;

- nie powinien zawierac sekretow ani danych wzraliwych;
- przy akcjach D3+ powinien tlumaczyc, co zostanie zapisane w audycie.

Katalog HelpTipow kluczowych ekranow

Execution Start

Element	Tresc/znaczenie
Header	Ekran prowadzi przez fazy 32-41: budowa, testy, predeploy, deploy/proba, zamkniecie.
Active project	Aktywny projekt jest kontekstem wykonania; wszystkie akcje zapisuja artefakty i audit dla niego.
Actions	Przyciski uruchamiaja kolejne fazy i po kazdej akcji odswiezaja projekt, acceptance i edge cases.
Operator notes	Notatka trafia do requestu jako kontekst zatwierdzenia; nie wpisywac sekretow.
Phase 32	Tworzy workspace, galezie, workerow, srodowiska i monitoring.
Phase 33	Uruchamia petle budowy i raportowanie kosztu/postepu/guardow.
Phase 34	Zwoluje rade w trakcie budowy przy zmianie zakresu lub ryzyku.
Phase 35	Wlacza koordynacje workerow, kolejki, blokady i recovery.
Phase 36	Waliduje artefakty, koszt, spojnosci i wygasza workerow.
Phase 37	Uruchamia L1-L5, coverage, performance i quality verdict.
Phase 38	Wystawia acceptance/staging i zbiera feedback klienta.
Phase 39	Sprawdza rollback, monitoring, support i hard authorization.
Phase 40	Wykonuje deploy albo lokalny release rehearsal; external wymaga Human Gate.
Phase 41	Tworzy raporty, archiwum, invoice, handoff i warranty.

Funding

Element	Tresc/znaczenie
Header funding	Granty, dotacje, auto-matching i generowanie wnioskow z masterplanu.
Zakladki	Firma, Nabory, Pomysly, Dopasowanie, Wnioski, Submission, Raporty.
Gotowosc	Procent uzupelnienia profilu; >80% rekomendowane przed wnioskami.
Nabory metric	Liczba aktywnych call'i zindeksowanych w systemie.
Wnioski metric	Liczba pakietow aplikacyjnych w przygotowaniu lub zlozonych.
Alerty metric	Aktywne alerty: terminy, brakujace dokumenty, walidacja.
KRS sync	Pobranie odpisu z API KRS i powiazanie ze sprawozdaniami.
Profil firmy	Dane prawne, finansowe i kompetencyjne wymagane przed matchingiem.
Nabory	Ewidencja programow, deadline'ow, budgetow i dokumentow.
Rekomendowane granty	Auto-generowane rekomendacje z engine dopasowania.
Pomysly	AI-generated ideas na podstawie kompetencji i naborow.
Projekty grantowe	Projekty z konsorcjum i dopasowaniami.
Scoring	Ocena szansy sukcesu na podstawie tematu, gotowosci, konsorcjum i historii.
Dopasowanie	Wyniki matchingu projektu do programow, score i uzasadnienie.
Wniosek	Generowanie pakietu z masterplanu, profilu i dokumentacji.
Bramka zlozenia	Walidacja dokumentow, review readiness i approval przed zlozeniem.
Sesja zlozenia	Walidacja, status zatwierdzenia i numer referencyjny.
Terminy	Alerty 30/14/7 dni przed deadlinem.
Alerty i zatwierdzenia	Brakujace dokumenty, bledy walidacji, historia decyzji.
Raport wykonawczy	Status pipeline, gotowosc, ryzyka, prognozowane przychody.

Project Start / Council / Planning

Ekran	Typ HelpTipow
/project-start	Wyjasnia faze aktywna, stan backendu, acceptance, edge cases, tworzenie projektu.
/council-to-ksiega	Wyjasnia role rady, pytania, consensus, generowanie ksiegi, finalizacje.
/planning	Wyjasnia model selection, skill synthesis, masterplan, test plan, cost preview i dry-run.

Globalne HelpTipy

Element	Znaczenie
HelpTip przy labelu	Opisuje pole albo metryke.
HelpTip przy sekcji	Tlumaczy caly panel.
HelpTip przy akcji	Tlumaczy skutki klikniecia.
HelpTip przy ryzyku	Tlumaczy, co blokuje lub wymaga Human Gate.

Legenda ikon i kontrolek

Kontrolka	Znaczenie
RefreshCw / OdŁwieŁŁ	Pobierz aktualny stan z backendu.
Rocket	Start fazy, deploy, uruchomienie.
ShieldCheck	Approval, guard, quality pass.
AlertTriangle	Blokada, alert, ryzyko.
Loader2	Akcja w toku.
Users	Rada, zespol, council.
GitBranch	Routing, decyzja, workflow.
TestTube	Testy i symulacje.
DollarSign	Koszt/funding/budzet.
Smartphone	Mobile/operator device.
Database	Memory, ontology, source of truth.
Badge zielony	Pass/gotowe/zaakceptowane.
Badge amber	W toku/oczekuje/ostrezenie.
Badge czerwony	Blad/blokada/niedostepne.
Tabs	Rozdzielaja workflow na kroki albo domeny.
Switch/checkbox	Wlaczanie polityki binarnej.
Select	Wybor projektu, aplikacji, sesji, modelu.
Text area	Notatka operatora, opis, uzasadnienie.

06 ROADMAP PRODUKCYJNA

Roadmapa produkcyjna AEIS 2026

Data: 2026-05-18

Status wyjściowy: **LOCAL_W14_ACCEPTANCE_PASS / DEV-STAGING CAPABLE / NOT PRODUCTION READY**

Cel: doprowadzić AEIS do wersji produkcyjnej bez przepisywania systemu od zera. Roadmapa zakłada konsolidację istniejących planes, domknięcie luk runtime, twardy hardening oraz testy prowadzone przez dashboard tak, jak robi to realny operator.

Spis treści

1. Werdykt startowy
2. Zasady nienegocjowalne
3. Mapa braków produkcyjnych
4. Roadmapa faz R0-R8
5. Backlog wykonawczy P0-P3
6. Testy akceptacyjne dashboardem
7. Zależności architektoniczne
8. Kryterium Production Ready

Werdykt startowy

AEIS ma działający lokalny runtime, dashboard operatorski, bogaty backend FastAPI, projektowy lifecycle, funding, test center, workers, skills, memory, Human Gate i liczne moduły laboratoryjne.

To nie jest jeszcze production-ready, ponieważ kluczowe planes są nadal pofragmentowane:

- storage produkcyjny nie jest wymuszony jako PostgreSQL;
- sekrety, backup, DR, rate limiting i secure headers nie są zamknięte jako twardy standard;
- Human Gate istnieje w kilku planes zamiast jednej prawdy decyzji;
- funding ma lokalny approval flow, a nie jednolity globalny ticket governance;
- memory, evidence i per-project runtime DB nie tworzą jednego write plane;
- skills registry, executor i demand signal nie są w pełni podpięte pod pipeline i execution;
- provider registry, model registry, council settings i routing modeli są nadal osobnymi planes;
- deployment produkcyjny ma rehearsal, ale wymaga pełnego cloud rollout, canary, rollback i DR drill;
- route-only powierzchnie dashboardu wymagają zamrożenia akcji, nie tylko renderu;
- mobile approval queue wymaga device binding, push i pełnego audytu urządzenia.

Najuczciwszy stan startowy:

AEIS działa jako silny system dev-staging i lokalny system testowy.
AEIS nie może być oznaczony jako production-ready, dopóki P0 i P1 z tej roadmapy nie przejdą 2X_PASS.

Zasady nienegocjowalne

Zasada 1: Stop, napraw, retestuj, zamrazaj

Komenda operatorska dla wszystkich dalszych prac:

Kazdy blad zatrzymuje flow.
Najpierw napraw przyczynę.
Potem wykonaj dwa pelne retesty tego samego flow.
Jezeli oba retesty przejdą, zapisz evidence, oznacz flow jako frozen i dopiero wtedy idz dalej.
Toast, sam route albo sam status 200 nie wystarcza.
Wymagane sa: efekt UI, efekt API, reload proof, audit/evidence i brak bledow konsoli.

Zasada 2: Route-action pair freeze

Kazda strona z akcja mutujaca musi miec:

- backend route **2X_PASS**;
- frontend action **2X_PASS**;
- obsluga bledow **network, 500, 403, timeout**;
- poprawna obsluga **204 No Content**;
- reload proof po zapisie;
- brak falszywego sukcesu w UI;
- wpis w bug ledger i freeze register.

Zasada 3: Jeden source of truth dla decyzji

Kazda decyzja D3+ musi przejsc przez unified Human Gate albo formalny Council flow. UI nie moze byc zrodlem prawdy autoryzacji. UI tylko pokazuje stan, a backend egzekwuje decyzje.

Zasada 4: Produkcja bez SQLite

SQLite zostaje tylko w **development**. **staging** i **production** wymagaja PostgreSQL 16+, migracji, backupu, restore drill i monitoringu polaczen.

Zasada 5: Evidence albo nie istnieje

Kazda naprawa, test, deploy, rollback, approval, external submit i freeze musi miec:

- **evidence_id**;
- source;
- checksum albo payload hash;
- operator/actor;
- timestamp;
- retention policy;
- link do artefaktu lub screenshotu.

Mapa brakow produkcyjnych

Obszar	Stan dzis	Brak produkcyjny	Ryzyko	Priorytet
Database	SQLite local/dev	PostgreSQL wymagany dla staging/prod, Alembic 2X, PgBouncer	utrata danych, brak concurrency, brak PITR	P0
Backup/DR	brak twardego drill	backup co 6h, restore 2X, RPO/RTO	brak odtworzenia po awarii	P0
Secrets	lokalne sekrety/dev	Vault, rotacja, audit sekretow	wyciek kluczy	P0
RBAC	czesciowo UX/API	backend enforcement na kazdym endpointzie mutujacym	bypass uprawnień	P0
Rate limit	brak globalnego standardu	Redis limiter per role i endpoint	abuse, runaway cost	P0
Secure headers	brak standardu freeze	HSTS, CSP, XFO, scan	ataki webowe	P1
Human Gate	wiele planes	jedna tabela, lifecycle, event sourcing	decyzje rozjezdzaja sie	P0
Funding submit	local gate	D4 global Human Gate, preview, receipt, audit	niechciany finalny submit	P0
Memory	federacyjna prawda	jeden write plane + project views	niespojna wiedza systemu	P1
Evidence	rozproszone artefakty	EvidenceSpine	brak audytu end-to-end	P1
Skills	registry/executor/demand split	pipeline i execution uzywaja skills automatycznie	manualne wywoływanie skills	P1
Model plane	registry/council/routing split	ModelControlPlane + BudgetEnforcer	niespojny routing modeli	P1
Workers	local smoke/live surface	zewnętrzna flota lifecycle 2X	brak realnej skalowalności	P1
Autoscaler	route istnieje	CPU/queue/error scale test	flapping albo brak reakcji	P2
Deploy	rehearsal	build, scan, staging, canary, prod, rollback	brak realnego rollout	P0
Testy	local W14 pass	CI golden/e2e/load/security	regresje po zmianach	P1
Mobile	queue surface	device binding, push, mobile audit	brak non-repudiation	P2
Global terminal	partial	actor/env/risk/rollback na kazdej komendzie	akcje bez właściciela	P2
Docs/onboarding	obszerne, ale stale rosnace	operator runbook + wizard < 15 min	bład operacyjny	P2

Roadmapa faz R0-R8

R0: Baseline produkcyjny i kontrakt zamrozenia

Cel: zamknac punkt startowy, zeby dalsze naprawy nie byly oceniane na podstawie starych raportow.

Deliverables:

- jeden dokument **PRODUCTION_BASELINE**;
- aktualny runtime health, OpenAPI count, frontend build, DB mode;
- lista planes, ktore sa source-of-truth, replica, legacy albo lab;
- zamrozony zestaw testow P1-P5 jako acceptance suite;
- jedna nomenklatura statusow: **BROKEN**, **PARTIAL**, **2X_PASS**, **FROZEN**, **PROD_READY**.

DONE:

- baseline zapisany w docs;
- dashboard smoke przechodzi;

- P0 backlog potwierdzony przez runtime, a nie tylko dokumentacje;
- wszystkie znane rozbieznosci maja ID backlogu.

R1: Foundation security, storage i operacje

Cel: usunac najwieksze ryzyka, zanim system zacznie wykonywac realne akcje zewnetrzne.

Zakres:

- wymusic PostgreSQL dla **staging** i **production**;
- zostawic SQLite tylko w **development**;
- dopiac migracje Alembic upgrade/downgrade 2X;
- wprowadzic backup, restore drill, RPO/RTO;
- przeniesc sekrety do Vault albo kompatybilnego secret managera;
- dodac backend RBAC enforcement dla endpointow mutujacych;
- dodac globalny rate limiter;
- dodac HSTS, CSP, X-Frame-Options i security scan.

DONE:

- aplikacja nie startuje w **production** na SQLite;
- migracje przechodza **dev -> staging -> prod** bez utraty danych;
- restore z backupu przechodzi dwa razy;
- kazdy endpoint mutujacy ma test roli;
- security scan bez critical vulnerabilities.

R2: Unified Human Gate i governance lifecycle

Cel: jedna prawda decyzji dla workspace, project, funding, deploy i mobile.

Zakres:

- **UnifiedHumanGateService**;
- tabela **human_gate_tickets**;
- **scope, decision_class, owner, status, audit_event_id**;
- event **HumanGateDecisionRecorded**;
- backend **require_role(...)** i **require_gate(...)** dla akcji D3+;
- adaptery: workspace, project, funding, mobile, deploy;
- jedna kolejka operatora;
- przejscia stanow: **pending, approved, rejected, escalated, expired**.

DONE:

- decyzja z **/human-gate** widoczna w project W18, funding i mobile;
- funding submit bez D4 approval jest niemozliwy;
- approval/reject dziala z dashboardu i mobile queue;
- audit log pokazuje actor, scope, D-class, payload hash i result;

- cross-plane flow przechodzi 2X.

R3: MemoryPlane i EvidenceSpine

Cel: system ma jedna pamiec zapisu i jeden kregoslup dowodowy.

Zakres:

- **MemoryPlane.write()** jako jedyny write path;
- **Entry { entry_id, content, provenance, project_id, evidence_id, created_by };**
- materialized **ProjectView**;
- **EvidenceSpine** dla freeze register, bug ledger, screenshots, API responses i audit jsonl;
- checksumy i retention policy;
- search zawsze filtruje po **project_id** i permissions;
- startup bootstrappuje memory indexer, evidence store i retrieval.

DONE:

- per-project DB nie przyjmuje primary writes poza MemoryPlane;
- kazdy wynik search ma provenance;
- **/memory/evidence/stats** dziala i nie jest shadowed przez dynamic route;
- memory/evidence przechodza API, UI, reload proof i 2X freeze.

R4: Skills + ModelControlPlane

Cel: skills i modele staja sie realna warstwa wykonawcza, a nie tylko katalog.

Zakres skills:

- runtime laduje skills z registry i filesystem podczas startupu;
- pipeline step moze wywolac skill jako executor;
- J5 dispatch moze wybrac skill zamiast raw worker;
- demand signal zmienia dispatch config albo tworzy backlog nowego skillu;
- wersjonowanie i rollback skills.

Zakres modeli:

- **ModelControlPlane**;
- **ProviderRegistry, ModelRegistry, CouncilConfig, RoutingTable, BudgetEnforcer**;
- council settings odwoluja sie do **ModelRegistry.model_id**;
- routing zawsze przechodzi przez budget/circuit breaker;
- key rotation propaguje sie do wszystkich consumers w < 60s.

DONE:

- flow **pipeline -> skill execution -> artifact -> evidence** przechodzi 2X;
- council voting uzywa modeli z registry;
- budget guard blokuje przekroczenie limitu;

- fallback chain jest testowany;
- route/model drift ma test regresyjny.

R5: FundingSubmissionGate i MobileOperatorPlane

Cel: realne akcje zewnętrzne są finalne tylko po mocnej bramce, a mobile ma tożsamość urządzenia.

Zakres funding:

- **PreSubmitCheck**: deadline, source, legal, budget, documents;
- preview pokazuje dokładnie payload/PDF, który zostanie wysłany;
- D3 dla local rehearsal, D4 dla real external submit;
- audit zapisuje **payload_hash, operator_id, timestamp, receipt**;
- jasny komunikat **No rollback after real submit**;
- CRM tracking po submit.

Zakres mobile:

- QR + TOTP device binding;
- push gateway dla pending D3+;
- approve/reject z device_id, PIN/biometric marker, timestamp;
- offline queue i sync;
- 375px viewport bez zoomu.

DONE:

- flow **profile -> call -> idea -> matching -> application -> preview -> gate -> submit -> receipt -> CRM** przechodzi 2X w trybie testowym;
- real submit ma mock/sandbox provider albo osobną zgodę operatora;
- mobile flow **ticket -> push -> open -> approve -> sync -> audit** przechodzi 2X.

R6: Production deploy, workers, autoscaler i DR

Cel: AEIS potrafi przejść pełny rollout i rollback w środowisku podobnym do produkcji.

Zakres:

- build Docker image;
- SBOM;
- vulnerability scan;
- staging deploy;
- smoke/golden tests na staging;
- canary 5% przez 15 minut;
- production rollout z circuit breaker;
- rollback automatyczny po error threshold;
- worker fleet: register, heartbeat, rebalance, graceful shutdown, evidence;
- autoscaler: CPU, queue depth, error rate;

- DR runbook i drill.

DONE:

- deploy i rollback przechodzą dwa razy;
- staging i production mają ten sam typ storage/secrets;
- rollback przywraca poprzednią wersję bez utraty danych;
- DR restore mieści się w RPO/RTO;
- runbook został wykonany przez operatora, nie tylko autora.

R7: Golden, E2E, load, security i self-test

Cel: regresja nie wraca po kolejnych zmianach.

Zakres:

- golden test suite dla frozen flows;
- dashboard E2E Playwright/Cypress dla P1-P5;
- flake rate < 2%;
- load test 10x expected peak;
- p99 < 500ms dla kluczowych endpoints;
- dependency scan i OWASP scan;
- dedicated autonomous skill lifecycle test;
- long-horizon memory/learning workflow test;
- AEIS self-test na własnym systemie.

DONE:

- CI blokuje merge przy regresji frozen flow;
- każdy P0/P1 fix ma test jednostkowy/integracyjny i dashboard retest;
- load/security raport zapisany jako evidence;
- finalny self-test generuje raport porównywalny z manualnym raportem.

R8: Runbook, onboarding i handoff

Cel: system może być obsługiwany przez nowego operatora.

Zakres:

- operator runbook: start, shutdown, incident, rollback, escalation;
- developer runbook: setup, migration, tests, release;
- onboarding wizard < 15 min;
- legenda zakładek i funkcji aktualna wobec runtime;
- help tips opisane i zgodne z akcjami;
- troubleshooting P0-P2;
- production acceptance checklist.

DONE:

- nowy operator przechodzi onboarding w < 15 min;
- runbook zostaje wykonany 2X;
- dokumentacja i dashboard nie roznia sie nazwami funkcji;
- PDF system book zostaje odswiezony po finalnym freeze.

Backlog wykonawczy P0-P3

P0: Blokery produkcji

ID	Zadanie	Zmiana	Test/FREEZE
PROD-P0-001	PostgreSQL required	production nie startuje na SQLite	migration upgrade/downgrade 2X
PROD-P0-002	Backup/restore	backup co 6h, restore drill	restore 2X, RPO/RTO proof
PROD-P0-003	Vault/secrets	secret manager + rotation	add/validate/rotate dummy secret 2X
PROD-P0-004	Backend RBAC	require_role na mutacjach	role matrix tests + bypass tests
PROD-P0-005	Unified Human Gate	jeden ticket lifecycle	cross-plane approval 2X
PROD-P0-006	Funding submit gate	D4 global gate	blocked submit + approved submit 2X
PROD-P0-007	Production deploy pipeline	staging, canary, prod, rollback	deploy + rollback 2X
PROD-P0-008	No false success	UI nie pokazuje sukcesu bez efektu backend	route-action freeze on priority pages

P1: Wysokie ryzyka architektoniczne

ID	Zadanie	Zmiana	Test/FREEZE
PROD-P1-001	MemoryPlane	centralny write path	write/search/reload/provenance 2X
PROD-P1-002	EvidenceSpine	evidence_id/checksum/retention	artifact aggregation 2X
PROD-P1-003	Skills integration	pipeline/execution trigger	pipeline -> skill -> artifact 2X
PROD-P1-004	ModelControlPlane	registry + routing + budget	council vote + fallback 2X
PROD-P1-005	Security headers	HSTS/CSP/XFO	browser/security scan 2X
PROD-P1-006	Rate limiting	Redis limiter per role	load/abuse test 2X
PROD-P1-007	Worker fleet	external lifecycle	register/heartbeat/rebalance/shutdown 2X
PROD-P1-008	Golden CI	frozen flows required in CI	fail-on-regression proof

P2: Produkcyjna dojrzalosc

ID	Zadanie	Zmiana	Test/FREEZE
PROD-P2-001	Mobile identity	device binding + push	mobile flow 2X
PROD-P2-002	Autoscaler	CPU/queue/error rules	scale up/down simulation 2X
PROD-P2-003	Global terminal policy	actor/env/risk/rollback	restart/rebuild/policy update replay 2X
PROD-P2-004	Route-only closure	priority pages action freeze	UI/API/reload/console proof
PROD-P2-005	Observability prod	logs/metrics/traces durable	incident replay proof
PROD-P2-006	Onboarding	wizard < 15 min	new operator 2X

P3: Uporzadkowanie i redukcja driftu

ID	Zadanie	Zmiana	Test/FREEZE
PROD-P3-001	Legacy dashboard boundary	oznaczyc jako legacy albo usunac z truth plane	docs/runtime sync
PROD-P3-002	Lab modules placement	cellular/SDR/devices/VPS jako labs	no production dependency proof

ID	Zadanie	Zmiana	Test/FREEZE
PROD-P3-003	Localization sweep	polski UI krytycznych powierzchni	visual pass
PROD-P3-004	Documentation drift check	nazwy docs == nazwy dashboardu	docs audit
PROD-P3-005	PDF refresh	ksiega po kazdym release candidate	render proof

Testy akceptacyjne dashboardem

Testy maja byc prowadzone jak przez czlowieka: dashboard, klikniecia, formularze, realne dane testowe, reload, screenshoty i weryfikacja API po akcji.

Zestaw projektow P1-P5

Projekt	Poziom	Cel testu	Produkcyjny dowod
P1 Mini CRM Local	latwy	prosty projekt bez zewnetrznych integracji	brak scope leak do platnosci/VPS/KSeF
P2 Funding Assistant	sredni	funding, dokumenty, blocked submit	D4 gate i audit przed submit
P3 Mobile Approval Queue	sredni+	approve/reject, kolejka, role	device/audit flow
P4 Local Automation Runtime	trudny	workers, retry, logs, local runtime	dispatch i observability
P5 Complex Multi-Domain	bardzo trudny	pelny AEIS spine	CRM + funding + mobile + runtime + governance

Minimalny scenariusz kazdego flow

1. Otworz dashboard.
2. Wprowadz dane recznie.
3. Kliknij akcje.
4. Sprawdz efekt UI.
5. Sprawdz efekt API.
6. Odswiez strone.
7. Sprawdz trwalosc danych.
8. Sprawdz audit/evidence.
9. Sprawdz brak bladov konsoli.
10. Powtorz flow drugi raz.
11. Dopiero wtedy wpisz freeze.

Co blokuje PASS

- route renderuje, ale przycisk nie robi realnej akcji;
- toast sukcesu bez zapisu;
- frontend ukrywa blad backendu;
- zapis znika po reload;
- brak audit trail dla decyzji D3+;
- Human Gate da sie ominac;
- funding submit nie ma preview i payload hash;
- memory zwraca wynik bez provenance;
- skill istnieje w registry, ale executor go nie zna;

- model council glosuje modelem spoza registry;
- deploy nie ma rollback proof.

Zaleznosci architektoniczne

flowchart TD

```

R0["R0 Baseline"] --> R1["R1 Security + Storage"]
R1 --> R2["R2 Unified Human Gate"]
R2 --> R3["R3 Memory + Evidence"]
R2 --> R5["R5 Funding + Mobile"]
R3 --> R4["R4 Skills + ModelControlPlane"]
R4 --> R6["R6 Deploy + Workers + DR"]
R5 --> R6
R6 --> R7["R7 Golden/E2E/Load/Security"]
R7 --> R8["R8 Runbook + Onboarding"]

```

Krytyczna kolejnosc:

1. Najpierw storage/security, bo bez tego testy produkcyjne nie maja sensu.
2. Potem Human Gate, bo funding, deploy i mobile musza uzywac jednej decyzji.
3. Potem Memory/Evidence, bo wszystko produkcyjne musi zostawiac dowod.
4. Potem Skills/ModelControlPlane, bo orchestration musi miec jeden registry/runtime truth.
5. Dopiero potem realny deploy, canary i rollback.

Kryterium Production Ready

AEIS moze dostac status **PRODUCTION READY** dopiero, gdy:

- wszystkie zadania P0 sa **FROZEN_2X**;
- wszystkie zadania P1 sa co najmniej **2X_PASS**, a krytyczne **FROZEN_2X**;
- P2 nie zawiera blockerow dla planowanego trybu produkcyjnego;
- staging i production dzialaja na PostgreSQL, nie SQLite;
- backup restore przeszedl dwa razy;
- Human Gate jest unified i backendowo egzekwowany;
- funding external submit wymaga D4 i ma finalny receipt/audit;
- deploy + rollback przeszedl w srodowisku production-like dwa razy;
- golden/e2e/load/security suite jest zielony;
- dashboard P1-P5 przeszedl jako czlowiek, z formularzami i danymi;
- nowy operator przeszedl onboarding i runbook;
- PDF system book zostal odswiezony po finalnym freeze.

Finalna decyzja musi byc wpisana jako evidence pack:

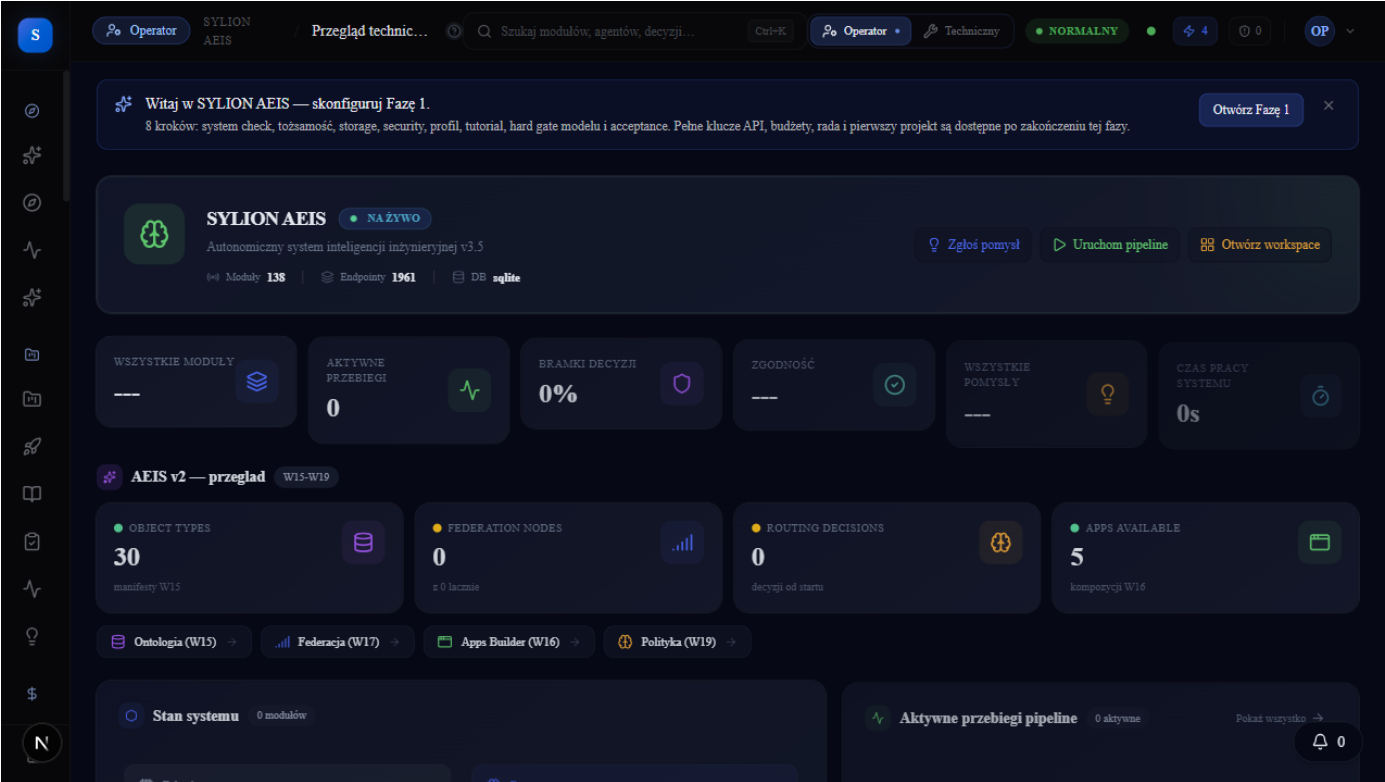
```

decision_class: D5
decision: AEIS_PRODUCTION_READY
required_approvals: operator + council + security + infra
rollback_plan: documented
evidence: baseline, freezes, scans, load test, DR drill, screenshots, runbook proof

```

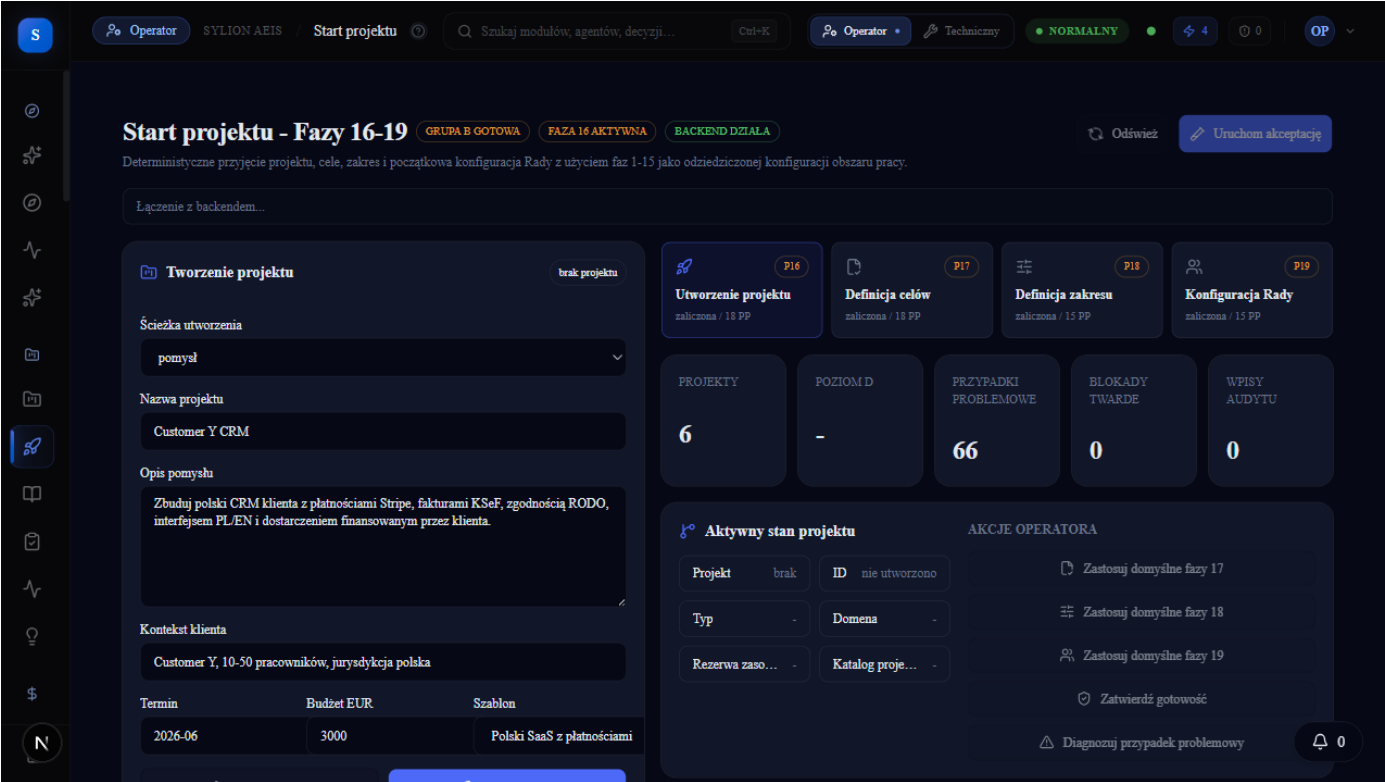
Galeria screenshotow runtime

00 overview



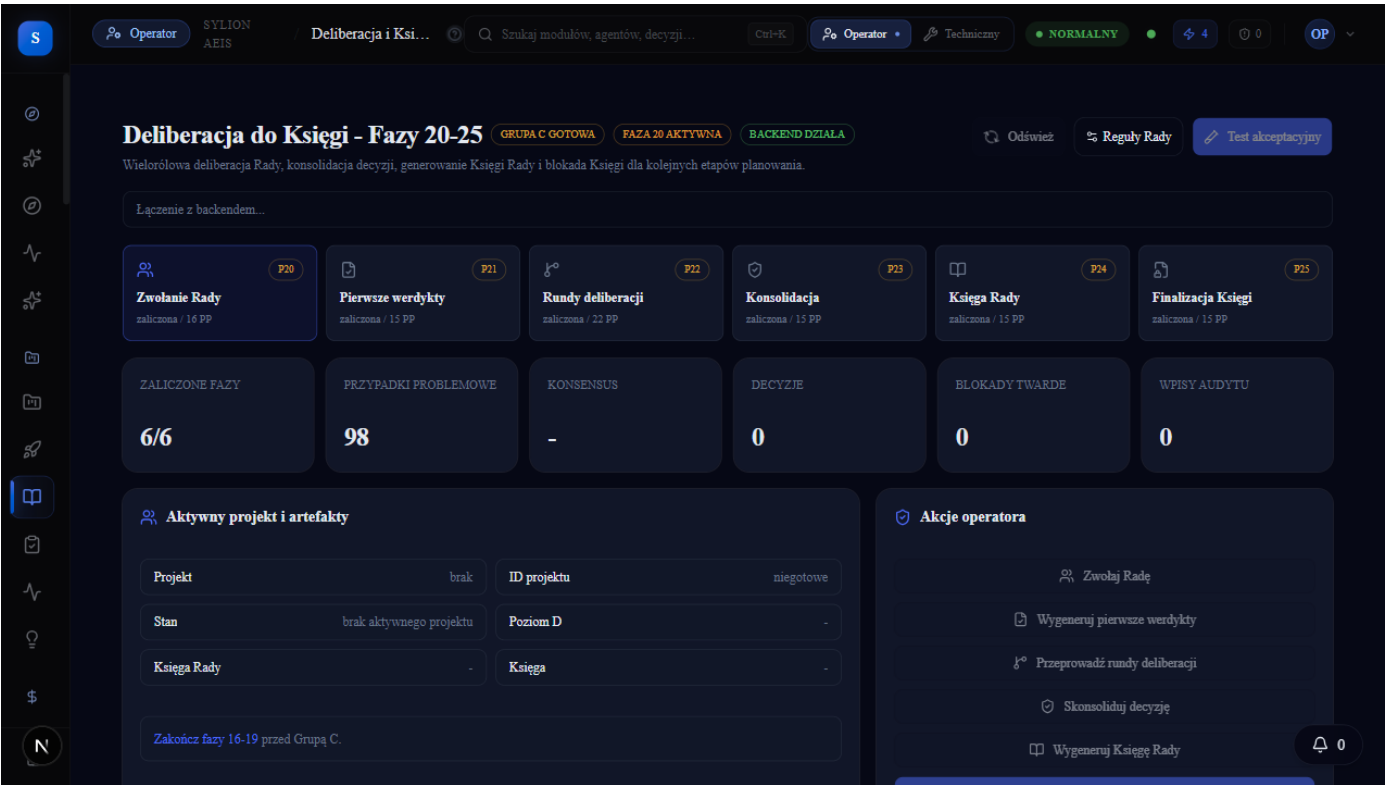
00_overview.png

01 project start



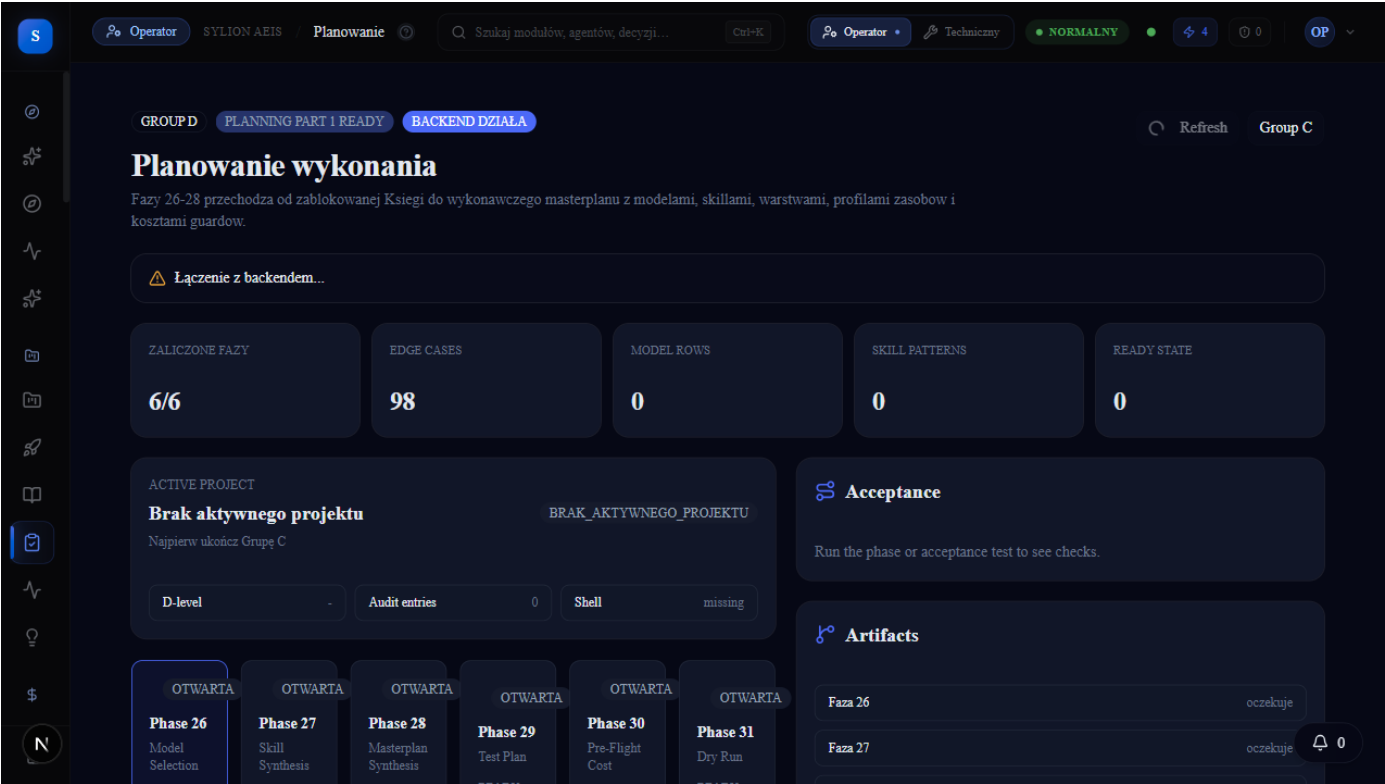
01_project_start.png

02 council to ksiega



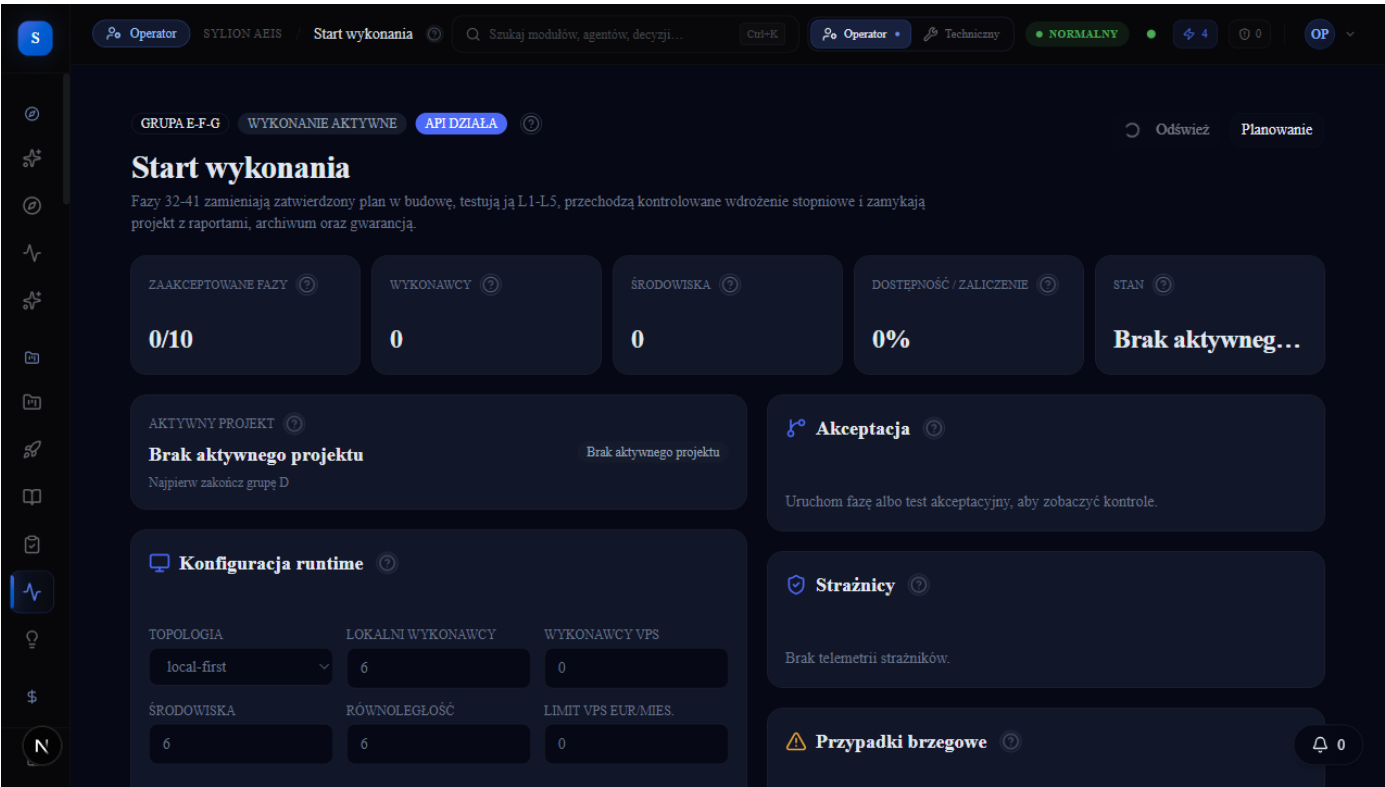
02_council_to_ksiega.png

03 planning



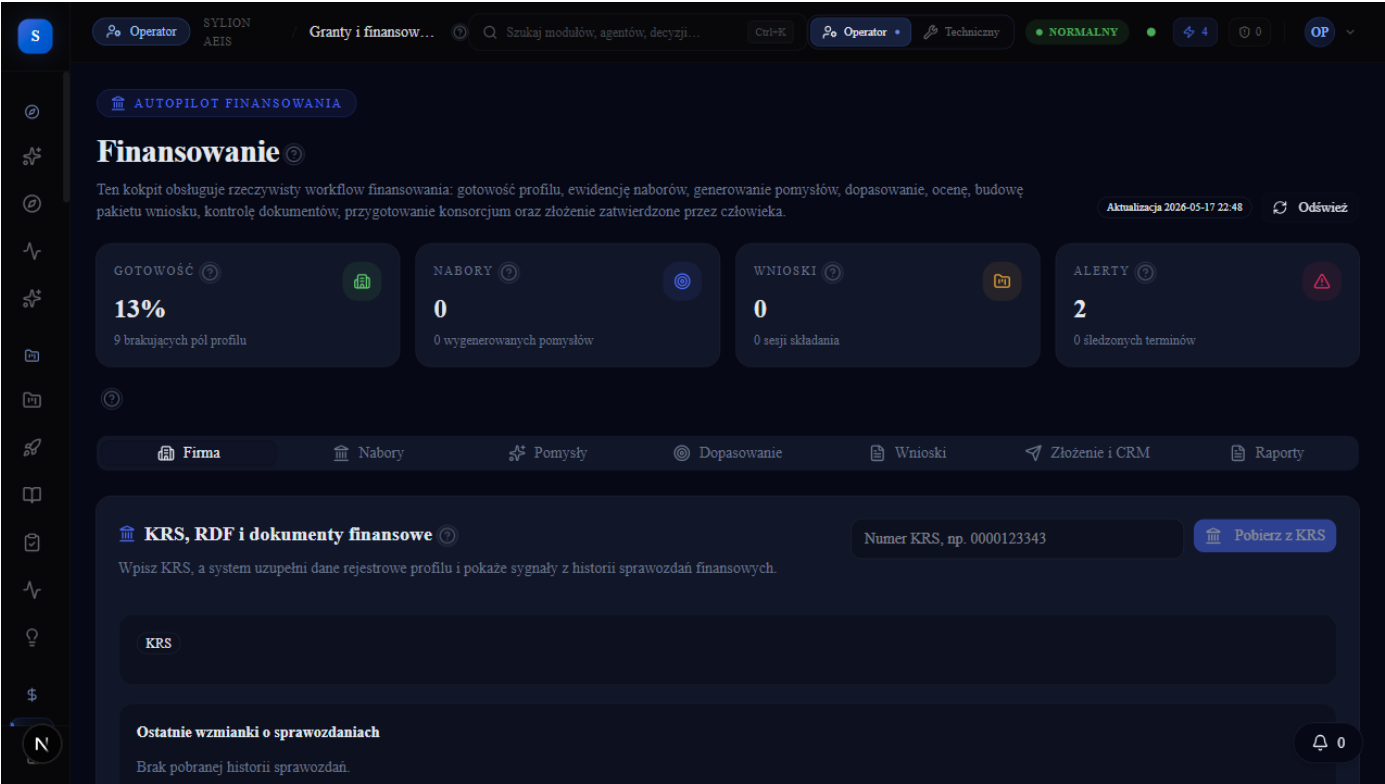
03_planning.png

04 execution start



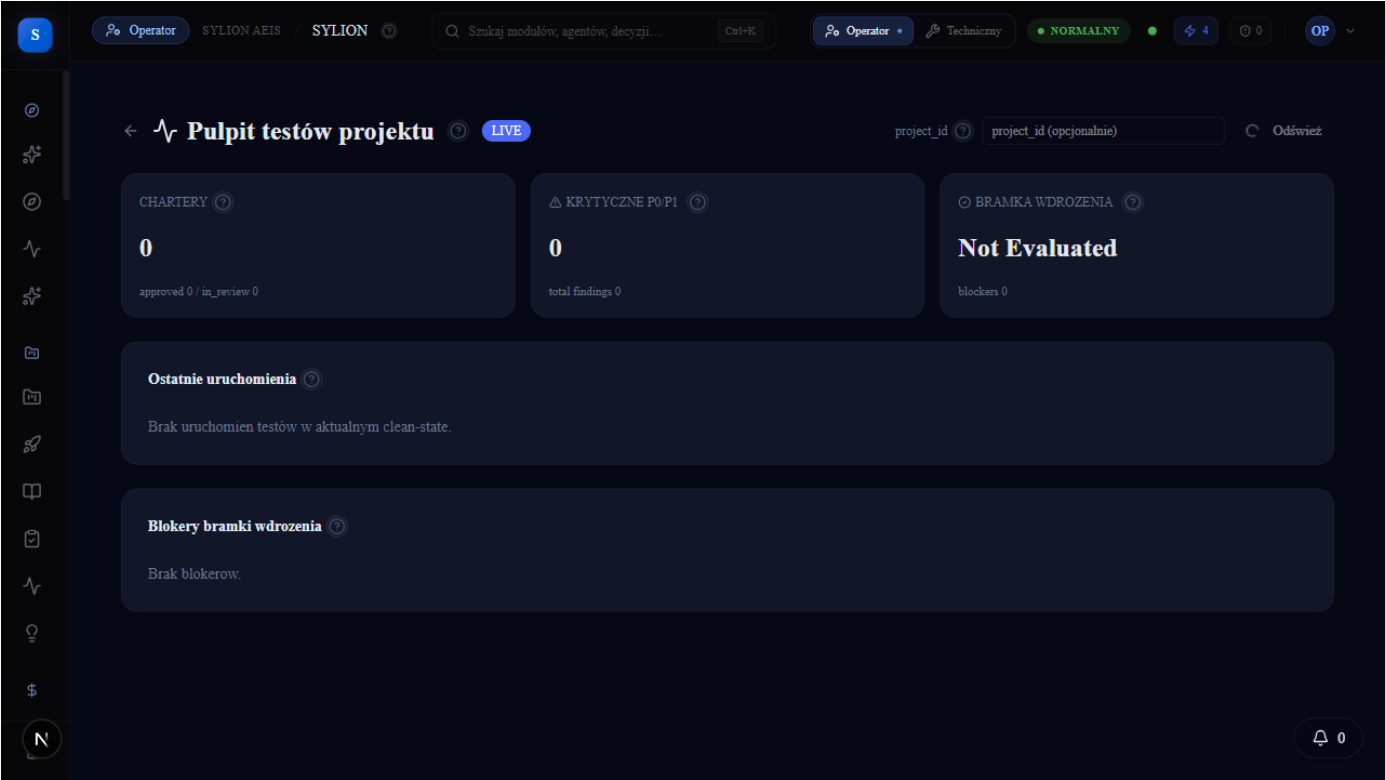
04_execution_start.png

05 funding



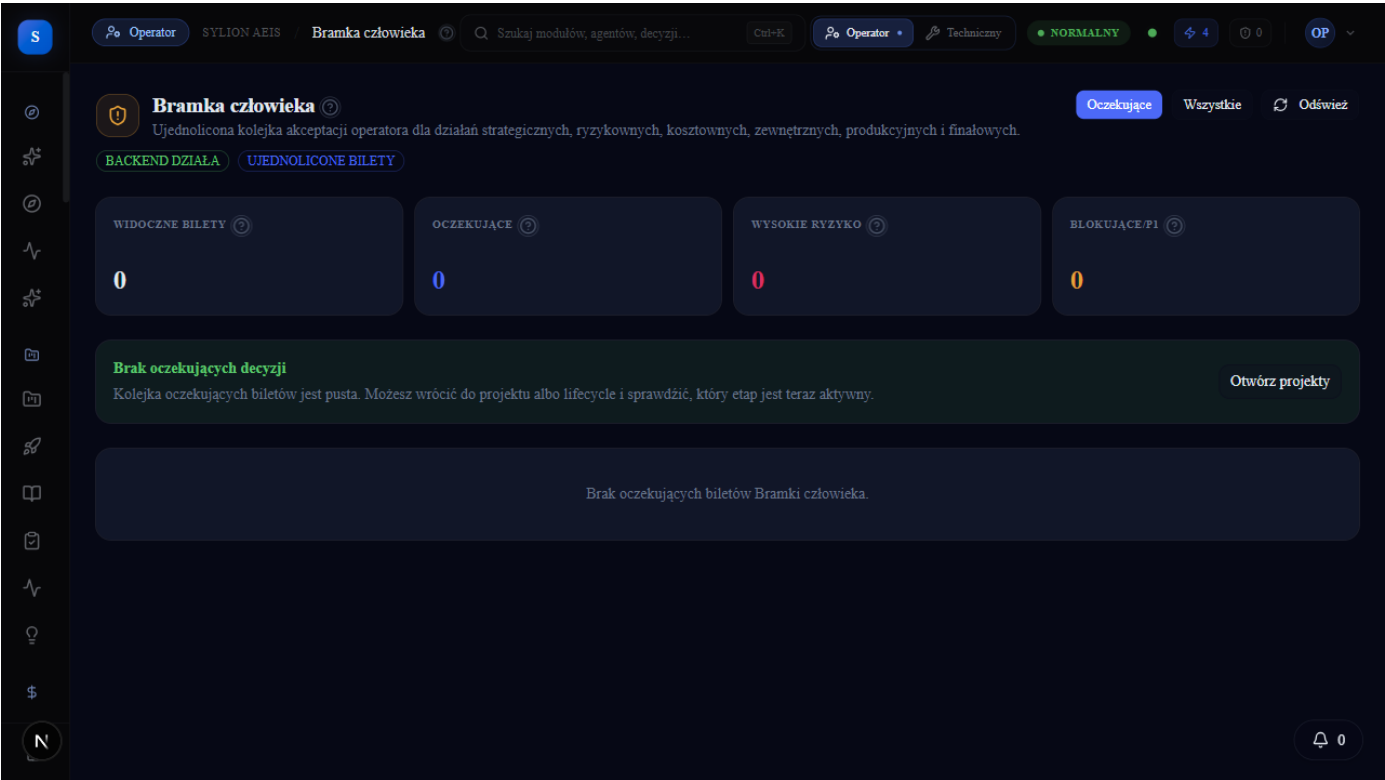
05_funding.png

06 test center



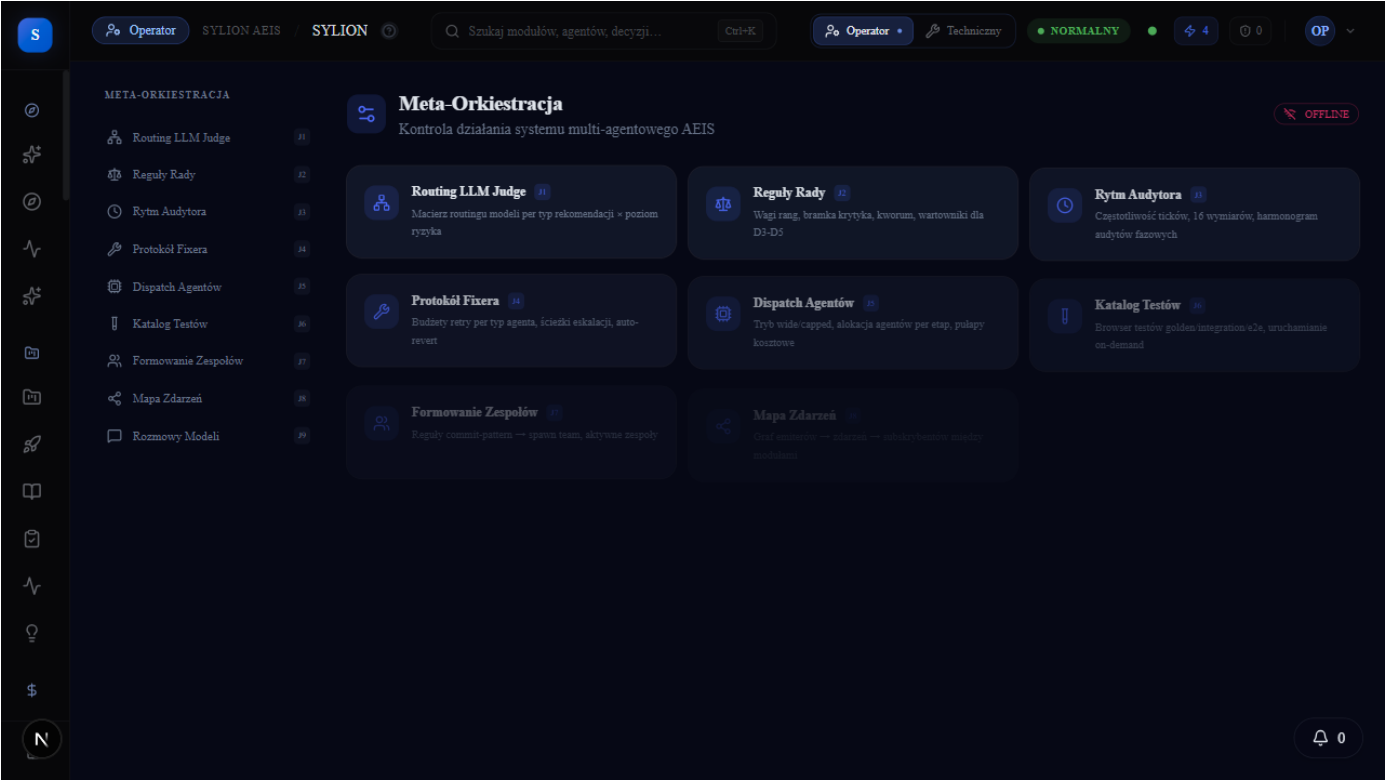
06_test_center.png

07 human gate



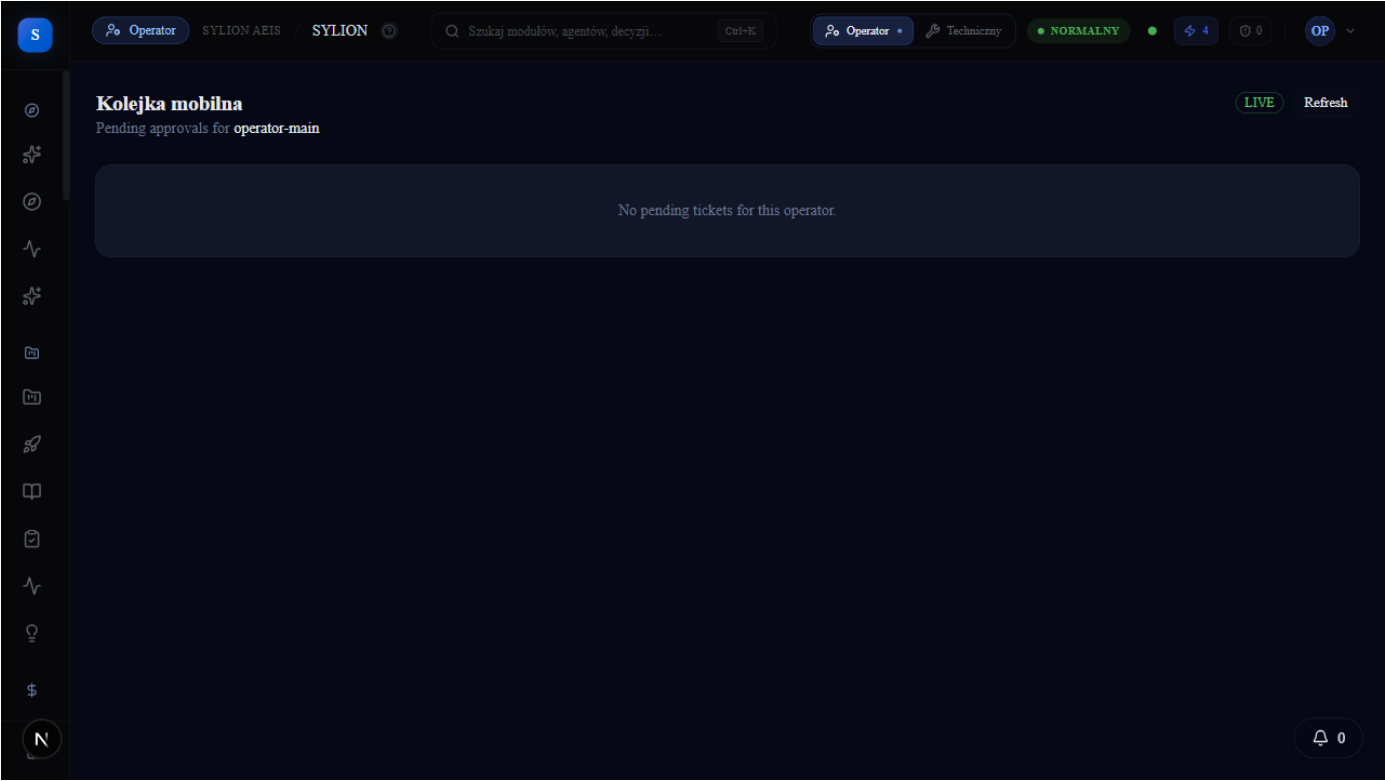
07_human_gate.png

08 orchestration



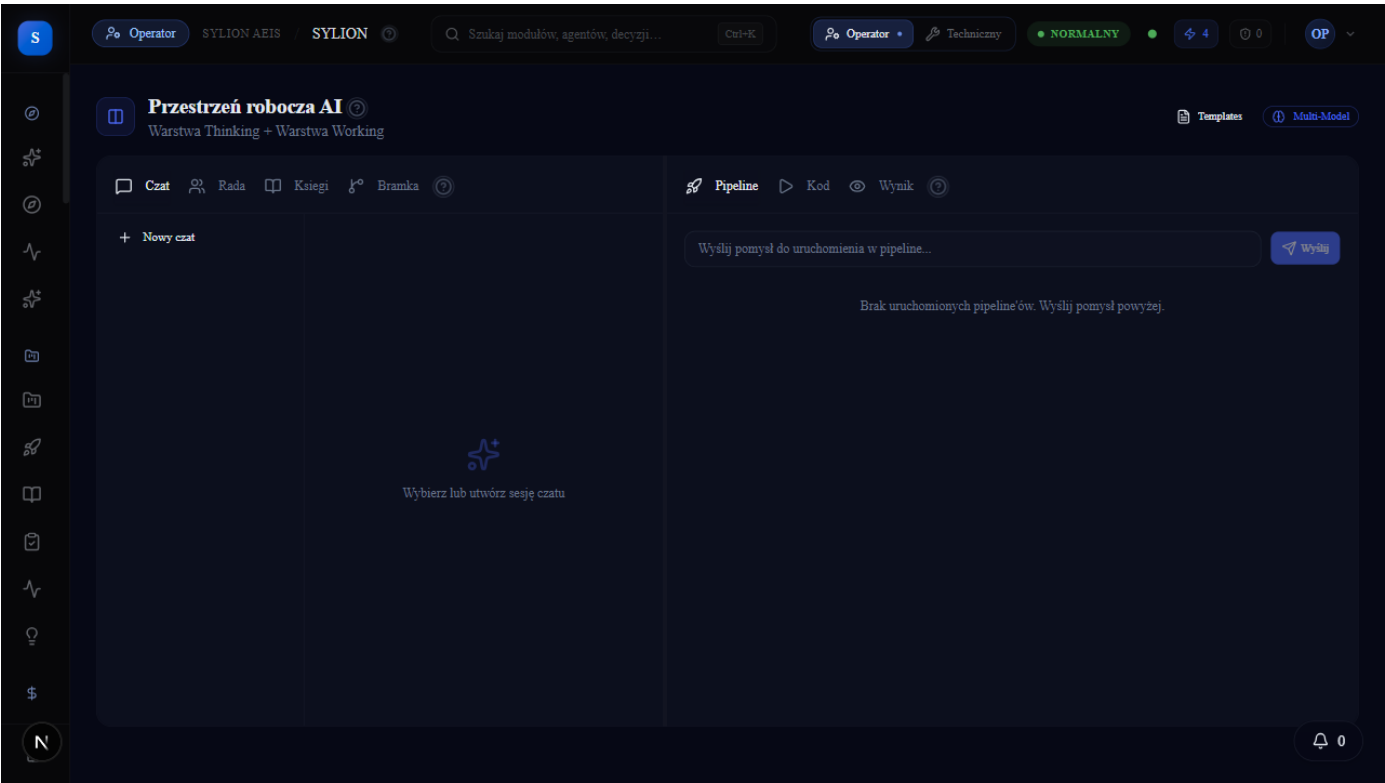
08_orchestration.png

09 operator mobile



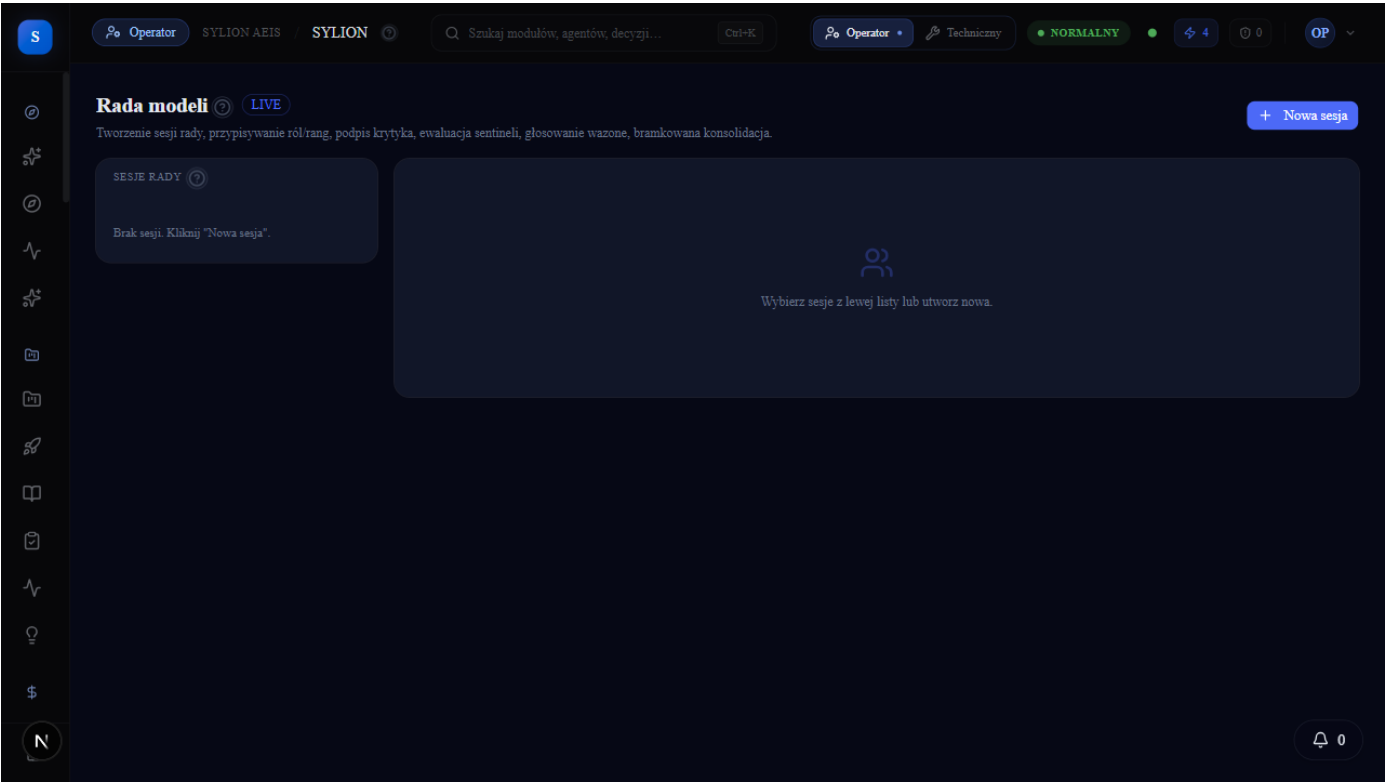
09_operator_mobile.png

10 workspace



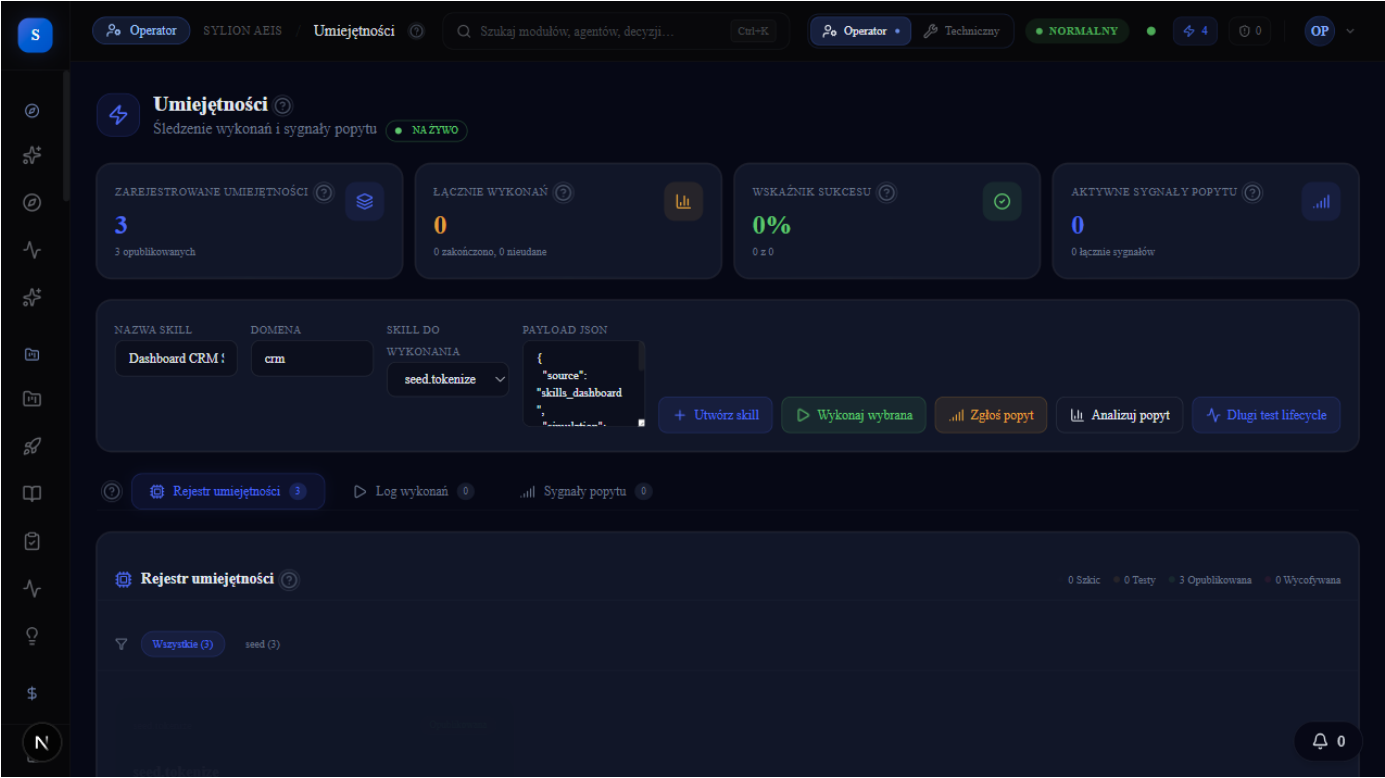
10_workspace.png

11 model council



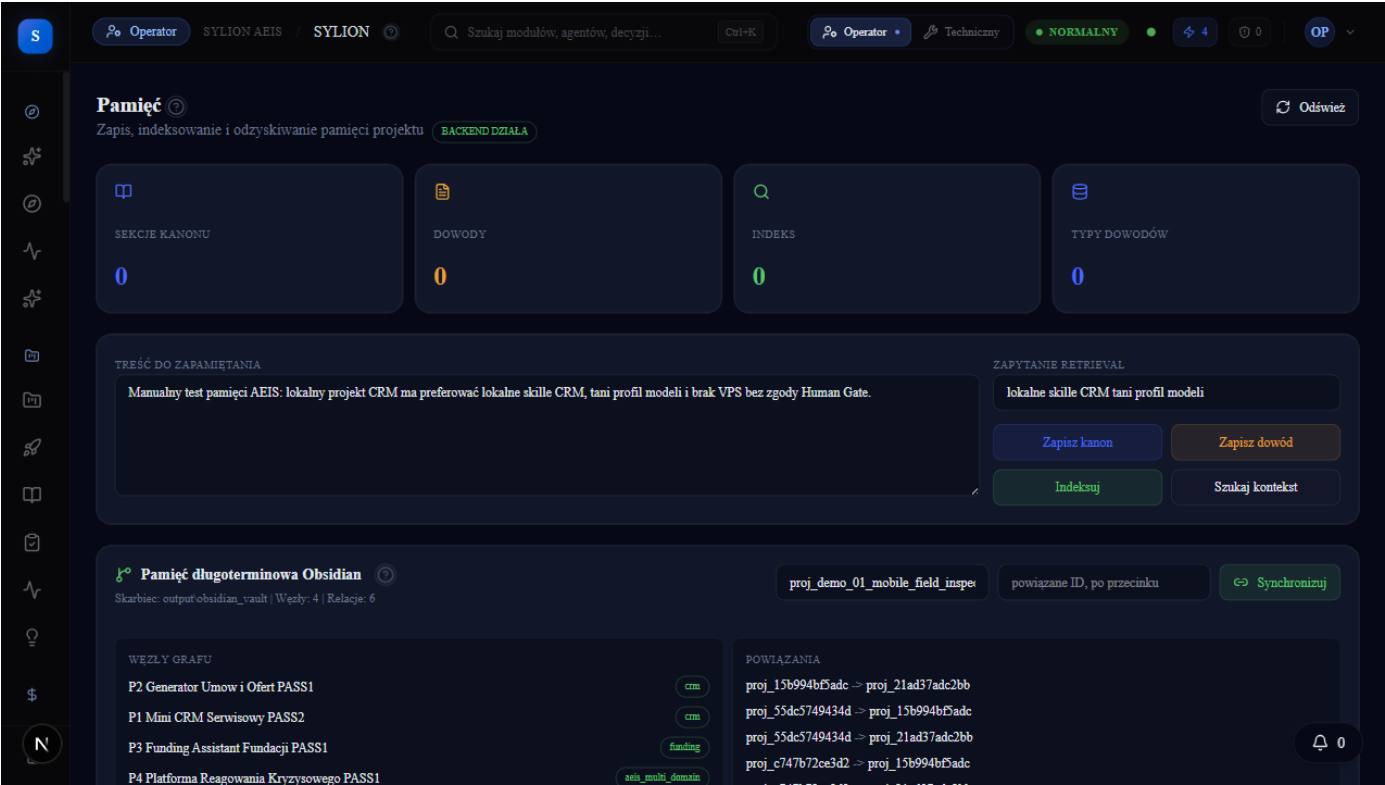
11_model_council.png

12 skills



12_skills.png

13 memory



13_memory.png